

# Efficient Size Constraint Community Search over Heterogeneous Information Networks

Xinjian Zhang<sup>†</sup>, Chengfei Liu<sup>†</sup>, Lu Chen<sup>†</sup>, Rui Zhou<sup>†</sup>, Bo Ning<sup>‡</sup>

<sup>†</sup>Swinburne University of Technology, Australia; <sup>‡</sup>Dalian Maritime University, China

<sup>†</sup>{xinjianzhang, cliu, luchen, rzhou}@swin.edu.au; <sup>‡</sup>ningbo@dlmu.edu.cn

**Abstract**—The goal of community search in heterogeneous information networks (HINs) is to identify a set of closely related target nodes that includes a query target node. In practice, a size constraint is often imposed due to limited resources, which has been overlooked by most existing HIN community search works. In this paper, we introduce the size-bounded community search problem to HIN data. Specifically, we propose a refined  $(k, \mathcal{P})$ -truss model to measure community cohesiveness, aiming to identify the most cohesive community of size  $s$  that contains the query node. We prove that this problem is NP-hard. To solve this problem, we develop a novel B&B framework that efficiently generates target node sets of size  $s$ . We then tailor novel bounding, branching, total ordering, and candidate reduction optimisations, which enable the framework to efficiently lead to an optimum result. We also design a heuristic algorithm leveraging structural properties of HINs to efficiently obtain a high-quality initial solution, which serves as a global lower bound to further enhance the above optimisations. Building upon these, we propose two exact algorithms that enumerate combinations of edges and nodes, respectively. Extensive experiments on real-world datasets demonstrate the effectiveness and efficiency of the proposed methods.

## I. INTRODUCTION

Heterogeneous information networks (HINs) comprise various types of nodes and edges, enabling them to effectively model real-world scenarios, including bibliographic, e-commerce, and social networks. As shown in Fig. 1(a), in a bibliographic network, nodes can represent entities such as authors, papers, topics, and venues. The relationships among these nodes include authors writing papers, papers having topics, and papers being published in venues. The structural representation of potential relationships between node types is called a schema, as illustrated in Fig. 1(b).

Analysing HIN data is fundamental yet critical. Among various analytic techniques, HIN community search, i.e., discovering correlated nodes of a given query node with the same node type (target type), has attracted significant attention. The HIN community search has a myriad of applications, such as recommendation [32], team formation [14], and identification of protein functions [6]. Most of the HIN community search methods [9], [12], [30] apply the meta-path ( $\mathcal{P}$ ) [24] to first establish virtual edges ( $\mathcal{P}$ -pairs) over target nodes, which forms a virtual graph ( $G_P$ ) based on  $\mathcal{P}$ -pairs. This step is necessary since most HINs focus on recording relations between nodes of different types (i.e., no relations exist between nodes of the same type). Then, classical cohesive subgraph models such as  $k$ -core [21], [22] and  $k$ -truss [5], [25] are applied to the

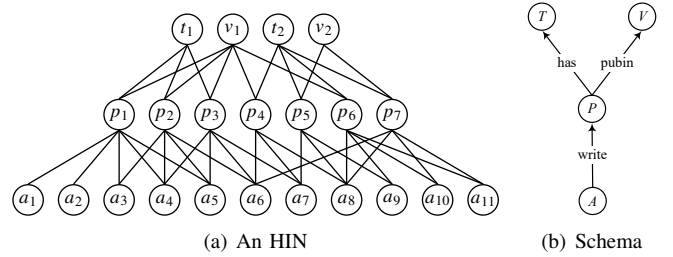


Fig. 1. An example of HIN with its schema

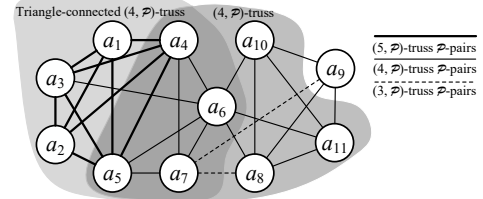


Fig. 2.  $G_P$  for Fig. 1(a) with  $\mathcal{P}=(A, P, A)$

virtual graph, which leads to popular HIN community models known as  $(k, \mathcal{P})$ -core and  $(k, \mathcal{P})$ -truss. What is more, the  $(k, \mathcal{P})$ -core model is then extended by considering other cohesiveness, such as spatial [2], [7], influence [37], etc., apart from target nodes' cohesiveness.

However, in many real applications, it is often necessary to impose a size constraint on the identified community due to budget or resource limitations. This motivates us to study the problem of community search in HIN with a size constraint. To the best of our knowledge, there is no existing work that focuses on this problem on HIN data.

**Our community model.** We first introduce a refined  $(k, \mathcal{P})$ -truss model, which serves as a cohesiveness measure. Then, we discuss the size constraint and the optimisation objective. **Triangle connected  $(k, \mathcal{P})$ -truss.** We adapt the mentioned  $(k, \mathcal{P})$ -truss to capture the cohesiveness of the target community. Given a query meta-path  $\mathcal{P}$ , if there is an instance of  $\mathcal{P}$  connecting two target nodes, then the two nodes are called a  $\mathcal{P}$ -pair. Given three target nodes, they form a triangle if any two of the nodes are in a  $\mathcal{P}$ -pair. A set of  $\mathcal{P}$ -pairs is called a  $(k, \mathcal{P})$ -truss if every  $\mathcal{P}$ -pair in the set forms at least  $k-2$  triangles with other  $\mathcal{P}$ -pairs in the set. Based on the definition, a query node  $q$  can be contained in only one connected community. This community may be formed by multiple dense parts, where some of these parts share exactly one node in common, i.e., they are 'weakly' connected. Such a structure may not be ideal for the community search problem. Therefore, to further

enhance the effectiveness of communities searched from a query node, we introduce the triangle connectivity into the  $(k, \mathcal{P})$ -truss model [30]. For a set of  $\mathcal{P}$ -pairs that is a  $(k, \mathcal{P})$ -truss, it further enforces that every two triangles either share one  $\mathcal{P}$ -pair or can be reached by a sequence of triangles s.t. the adjacent triangles in the sequence share one  $\mathcal{P}$ -pair. By doing so, a single query node can be part of multiple communities, which can be denser, making the community model more effective.

*Size constraint and optimisation objective.* As discussed, existing community search problems for HIN data do not account for the size constraint that naturally exists in real applications, and they are likely to maximise community size. In contrast, we introduce a size constraint  $s$  to limit the number of nodes forming the triangle-connected  $(k, \mathcal{P})$ -truss. Accordingly, we take cohesiveness as the optimisation objective, i.e., we aim to find the maximum possible  $k^*$  such that there exists a triangle connected  $(k^*, \mathcal{P})$ -truss containing the query node. We shorten our studied problem, i.e., the **Size-constraint Community Search in HIN** maximising cohesiveness, to the **SCSH** problem.

**Applications.** As discussed, the SCSH problem has many real-world applications. Below, we present two of them.

Suppose we want to discover a panel using the DBLP dataset; we may consider strong collaborative authors as candidates, given a core panel member (author). To achieve that, we need to establish collaboration relationships among authors first. This can be achieved by using the meta-path (*Author, Paper, Author*), which naturally spans multiple research groups, as authors from different teams may collaborate. Based on the derived relations,  $k$ -truss ensures that the authors have strong collaborative relations, and the triangle connectivity further enhances these collaborative relations. Besides, a panel is typically invite-only and has a strictly bounded number of participants due to format and venue constraints. As such, our model identifies a panel that includes members from multiple collaboration circles while maintaining sufficient collaboration coherence during panel discussions.

Suppose we want to recommend items to users with an interested or a purchased item  $i$  using the Amazon dataset, we may want to discover items similar to  $i$  as candidates. We establish relations over items via a meta-path, such as (*Item, View, Item*), which reflects co-view-based semantics popular for recommendation tasks. In the derived graph, the most cohesive  $k$ -truss containing  $i$  ensures that the items are highly similar to  $i$ , while the triangle connectivity further strengthens this. However, such a cohesive subgraph may contain too many items, which may confuse or overwhelm users. Therefore, the size constraint in our model ensures a fixed number of highly relevant items, simplifying decision-making and enhancing the overall user experience.

Our model identifies tightly connected, size-bounded communities for such scenarios.

**Technical challenges and state-of-the-art.** We prove that the SCSH problem is NP-hard, which is indeed very different from the well-studied  $(k, \mathcal{P})$ -truss problem without a size constraint.

There are related works on homogeneous networks that address the most cohesive  $k$ -core [31] and  $k$ -truss [34] problems. Both approaches solve the problems using an inclusion-and-exclusion-based node set generation framework equipped with corresponding optimisations such as branching, bounding, and candidate set reduction techniques. [31] is advanced in terms of devising upper bounds, as the node set generation-based framework naturally supports cohesiveness defined on nodes more effectively. Due to the difficulty of devising effective upper bounds, [34] is more advanced in terms of devising lower bounds.

**Technical gaps.** Although we can get ideas from [31] and [34], we find the following technical gaps. First, all of them are dedicated to homogeneous networks. In fact, HINs possess certain desirable properties that can accelerate the search, which will be explored in this paper. Second, existing works use an inclusion-and-exclusion-based node set generation framework, which is efficient for generating all combinations of  $n$  nodes. However, our problem only requires exploring  $s$ -size combinations of  $n$  nodes, and practically,  $s$  is far less than  $n$ . The inclusion-and-exclusion-based node set generation leads to a recursion depth of  $n$  rather than  $s$ . We need a more efficient way of doing so. Last but not least, even the most closely related work [34] does not consider triangle connectivity when devising algorithms, which could be improved.

**Our solution.** To fill the above research gaps, we present a novel branch-and-bound algorithm for the SCSH problem based on the lexicographic generation framework. This naturally bounds the recursion depth by  $s$ . This improvement enables both edge set and node set enumeration approaches. For bounding techniques, we first take advantage of the HIN property, i.e., taking polynomial time to get large pairwise connected  $\mathcal{P}$ -pairs, to derive near-optimum results, as a lower bound. This lower bound significantly reduces the candidate size. We then devise novel and effective upper bounds for both node set and edge set enumeration approaches, considering triangle connectivity. For the edge set enumeration, we propose an upper bound by enlarging the trussness of the edge in the partial result with the minimum trussness, considering the maximum triangle-connected triangles that can be included from the candidate set. However, the above upper bound cannot be applied to the node set enumeration approach. This is because, for node set enumeration, we can remove some edges with low trussness from the node set induced subgraph to improve the result, i.e., deriving an upper bound on edges with minimum trussness does not make sense here. We alternatively seek to increase the trussness of the high trussness edges in the partial result, in order to satisfy the necessary condition for improving the overall trussness and to derive a reasonable upper bound. Using the lower and upper bounds, we can effectively and efficiently bound the search towards the optimum result. Furthermore, we propose a novel total search order and branching strategy. Together with all these optimisations, the edge set generation based algorithm runs in  $\mathcal{O}(|N(q, G_P)| \binom{|E(G)|}{(s-1)(s-2)} |E(G_P)|^{1.5})$  with the most advanced

upper bounds and the node set generation based algorithm runs in  $\mathcal{O}(|N(q, G_P)| \binom{|V(G_P)|}{s-2})|E(G_P)|^{1.5}$  with a reasonable upper bound, where  $G_P$  is the virtual graph consisting of all  $\mathcal{P}$ -pairs in HIN. In practice, the node set generation approach runs one order of magnitude faster than the edge set generation approach, and the edge set generation approach is, on average, 44% faster than the method adapted from SOTA.

Our principal contributions are summarized as follows.

- We introduce the problem of size-bounded community search in heterogeneous information networks, maximizing the cohesiveness. **Section II**
- We introduce a novel search framework that brings low time complexity for both edge and node set enumeration based methods. **Section IV-A**
- We develop a novel heuristic algorithm that efficiently identifies a feasible community, serving as a tighter lower bound. **Section IV-B**
- We propose novel bounding, branching, total order, and reduction optimisations, which lead to two efficient exact algorithms. **Sections IV-C to IV-F**
- We validate our approaches through extensive experiments on real-world datasets, demonstrating their effectiveness and efficiency. **Section V**

## II. PROBLEM FORMULATION

### A. Preliminaries

**Heterogeneous Information Network (HIN).** An HIN is a directed graph  $H(V, E)$  with a node type mapping function  $\phi: V \rightarrow \mathcal{A}$  and an edge type mapping function  $\psi: E \rightarrow \mathcal{R}$ , in which every  $v \in V$  has a node type  $\phi(v) \in \mathcal{A}$  and every  $e \in E$  has an edge type  $\psi(e) \in \mathcal{R}$ .

**HIN Schema.** Given an HIN  $H=(V, E)$  with mappings  $\phi: V \rightarrow \mathcal{A}$  and  $\psi: E \rightarrow \mathcal{R}$ , its schema is a graph defined over node types  $\mathcal{A}$  and edge types  $\mathcal{R}$ , denoted as  $T_H(\mathcal{A}, \mathcal{R})$ .

Fig. 1(b) shows the DBLP schema with node types  $\mathcal{A}$ ,  $\mathcal{P}$ ,  $\mathcal{T}$ , and  $\mathcal{V}$  (authors, papers, topics, venues) and reversible edges (e.g.,  $A-P$  denotes “write” while  $P-A$  denotes “written by”).

**Meta-path.** A meta-path  $\mathcal{P}$  is a path defined on a schema  $T_H=(\mathcal{A}, \mathcal{R})$ , and is denoted in the form  $A_1 \xrightarrow{R_1} A_2 \xrightarrow{R_2} \dots \xrightarrow{R_l} A_{l+1}$ , where  $l$  is the length of  $\mathcal{P}$ ,  $A_i \in \mathcal{A}$ , and  $R_i \in \mathcal{R} (1 \leq i \leq l)$ . For simplicity, we denote  $\mathcal{P}$  as an ordered set in forms of  $(A_1, A_2, \dots, A_{l+1})$ .

Given that relationships in an HIN are reversible, meta-paths are thus reversible. If reversed meta-path  $\mathcal{P}^{-1}$  equals  $\mathcal{P}$ , it is symmetric (e.g.,  $(A, P, A)$ ). In this paper, all meta-paths are assumed symmetric.

**Instance of meta-path.** Given a path  $p=a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_{l+1}$  and a meta-path  $\mathcal{P}=(A_1, A_2, \dots, A_{l+1})$ ,  $p$  is an instance of  $\mathcal{P}$ , if  $\forall i \in [1, l]$ , the node  $a_i$  and edge  $e_i=(a_i, a_{i+1})$  satisfy  $\phi(a_i)=A_i$  and  $\psi(e_i)=R_i$ .  **$\mathcal{P}$ -pair and  $G_P$ .** The source and sink nodes of a  $\mathcal{P}$  instance form a  $\mathcal{P}$ -pair. For example, in Fig. 1(a), the path  $a_1 \rightarrow p_1 \rightarrow a_4$  is an instance of the meta-path  $(A, P, A)$ , and  $(a_1, a_4)$  is a  $\mathcal{P}$ -pair. After identifying all  $\mathcal{P}$ -pairs, we can construct a **derived homogeneous graph, denoted as  $G_P$** . Fig. 2 illustrates an example of  $G_P$  derived from Fig. 1(a) based on  $(A, P, A)$ .

TABLE I  
FREQUENTLY USED NOTATIONS

| Notation             | Definition                                                                       |
|----------------------|----------------------------------------------------------------------------------|
| $\mathcal{P}$        | A meta-path                                                                      |
| $G_P$ , $G_P[\cdot]$ | Graph consists of $\mathcal{P}$ -pairs, nodes (edges) induced subgraph of $G_P$  |
| $C, R$               | Partial result, candidate set (a set of edges or nodes)                          |
| $C^*$                | Global optimum results or optimum result up to the time                          |
| $UB(C, R)$           | The upper bound of triangle-connected trussness for the recursion state $(C, R)$ |
| $C \leq \tau$        | The set of edges that trussness is not greater than $\tau$                       |
| $E_{min}$            | The minimum trussness among the edges in $C$                                     |
| $ \Delta_{opt} $     | The optimistic number of capable triangles                                       |
| $ E_{opt} $          | The optimistic number of edges                                                   |
| $\Phi_E(u)$          | The set of nodes dominated by $u$ based edges                                    |
| $\Phi_\Delta(u)$     | The set of nodes dominated by $u$ based triangles                                |

**Triangle ( $b$ -triangle in [30]).** Given an HIN  $H$  and a meta-path  $\mathcal{P}$ , a triangle  $\Delta$  is a triplet of nodes  $(u, v, w)$  such that every pair of nodes in  $\Delta$  forms a  $\mathcal{P}$ -pair.

In [30], two types of triangles are discussed according to the constraints over the instances of  $\mathcal{P}$  forming a triangle. In this paper, we focus more on  $b$ -triangles and will call them triangles from now on for brevity.

**Support.** Given a set  $S$  of  $\mathcal{P}$ -pairs for  $H$ , the support of a  $\mathcal{P}$ -pair  $(u, v)$  regarding  $S$  is the number of triangles, which are formed by  $\mathcal{P}$ -pairs in  $S$ , containing  $(u, v)$ .

**Definition 1.  $(k, \mathcal{P})$ -truss [30]:** Given a set  $S$  of  $\mathcal{P}$ -pairs for  $H$ , the  $(k, \mathcal{P})$ -truss is the maximum set of  $\mathcal{P}$ -pairs, denoted by  $S'$ , such that for each  $\mathcal{P}$ -pair, its support in  $S'$  is at least  $k-2$ .

**Definition 2. Trussness of  $\mathcal{P}$ -pair:** The trussness of a  $\mathcal{P}$ -pair  $(u, v)$  regarding  $S$ , denoted by  $\tau_S(u, v)$ , is the largest  $k$  such that  $\mathcal{P}$ -pair  $(u, v)$  is contained in a  $(k, \mathcal{P})$ -truss of  $S$ .

**Definition 3. Trussness of node:** The trussness of a node  $v$  regarding  $S$ , denoted by  $\tau_S(v)$ , is the maximum trussness among all  $\mathcal{P}$ -pairs containing  $v$ , i.e.,  $\tau_S(v) = \max\{\tau_S(u, v) | (u, v) \in S\}$ .

### B. Problem formulation

Based on the  $(k, \mathcal{P})$ -truss, we further introduce the well-studied concepts of triangle adjacency and triangle connectivity to enhance the efficacy of  $(k, \mathcal{P})$ -truss and then introduce the size constraints, which lead to our studied SCSH problem.

**Triangle adjacency.** Two triangles,  $\Delta_s$  and  $\Delta_t$ , are adjacent if they share a  $\mathcal{P}$ -pair, i.e.,  $|\Delta_s \cap \Delta_t| = 2$ .

**Triangle connectivity.** Two triangles,  $\Delta_s$  and  $\Delta_t$ , are triangle connected, if there exists a sequence of triangles in  $S$ ,  $\Delta_s, \dots, \Delta_n$  ( $n \geq 2$ ) such that for  $1 \leq i < n$ ,  $|\Delta_i \cap \Delta_{i+1}| = 2$  and  $\Delta_1 = \Delta_s$ ,  $\Delta_n = \Delta_t$ .

**Definition 4. Triangle-connected  $(k, \mathcal{P})$ -truss:** A set of  $\mathcal{P}$ -pairs  $S$  is a triangle-connected  $(k, \mathcal{P})$ -truss if it satisfies: (1)  $S$  is a  $(k, \mathcal{P})$ -truss, and (2)  $\forall \Delta_1$  and  $\Delta_2 \in S$ ,  $\Delta_1$  is triangle connected with  $\Delta_2$  within  $S$ .

**SCSH problem.** Given an HIN  $H(V, E)$ , a query node  $q$ , a community constraint size  $s \in \mathbb{N}^+$ , and a meta-path  $\mathcal{P}$ , i.e. the input is  $(H, \mathcal{P}, q, s)$ . The objective is to find a community  $G'_P=(V^*, S^*)$  (formed by a set of  $\mathcal{P}$ -pairs), which satisfies the following conditions:

- 1) Query awareness:  $q \in V^*$ ;
- 2) Size constraint:  $|V^*| = s$ ;

- 3) Cohesiveness maximisation:  $G'_P$  is a triangle-connected  $(k, \mathcal{P})$ -truss and has the maximum  $k$  compared to any other  $G''_P$  meeting the above two conditions.

An example has been provided in the introduction. And the notations that are frequently used are summarised in Table I. Next, we analyse the hardness.

**Theorem 1.** SCSH problem is NP-hard.

*Proof.* We prove that the decision version of the SCSH problem is NP-complete via a reduction from the  $m$ -clique problem, which is known to be NP-complete.

The  $m$ -clique problem takes a graph  $G=(V,E)$  and an integer  $m$  as input, and asks whether  $G$  contains a clique of size at least  $m$ . The decision version of the SCSH problem changes the optimisation objective (maximising the trussness) to a constraint, i.e.,  $C$  is a triangle-connected  $(k, \mathcal{P})$ -truss for a given  $k$  (other constraints are identical to the SCSH problem).

We first present a gadget that can transform  $G$  into an instance of HIN ( $H$ ) with two types of nodes ( $T_1$  and  $T_2$ ) and a network schema consisting of just one undirected edge  $(T_1, T_2)$ . The gadget is as follows. For each  $e_i(u, v)$  in  $E(G)$  ( $1 \leq i \leq |E(G)|$ ), we create node  $u, v$  with type  $T_1$  in  $H$  if  $u$  and  $v$  are not in  $H$  yet (we do not create an edge between  $u$  and  $v$  in  $H$ ). Then, we create a new dummy node  $w_i$  with type  $T_2$  for  $e_i$ . After, we create two undirected edges  $(u, w_i)$  and  $(v, w_i)$  in  $H$ . At last, we create an extra dummy node  $q$  of type  $T_1$  and link it to each created  $w_i$ . Clearly, applying the gadget takes linear time w.r.t.  $|E(G)|$ .

By applying the gadget, the instance of our problem is as follows.  $H$  is generated by the gadget,  $\mathcal{P}$  is  $(T_1, T_2, T_1)$ ,  $s$  and  $k$  are set to be  $m+1$  (notice that a size  $s$   $s$ -truss is a  $s$ -clique), and  $q$  is query node.

It is easy to see that  $\mathcal{P}$ -pairs form a graph  $G_P$  that contains the identical structure as  $G$  in the  $m$ -clique problem, with an extra node  $q$  connecting to every node in  $G$ . This means that each size  $m$ -clique in  $G$  corresponds to an  $m+1$ -clique in  $G_P$ . In reverse, every  $m+1$ -clique in  $G_P$  (containing  $q$ ) leads to a  $m$ -clique in  $G$  by removing  $q$ . It is clear to see that computing  $G_P$  for the given  $H$  takes  $p$ -time.

We show that the above instance of the decision version of our studied problem is at least as hard as the  $m$ -clique problem, i.e., the decision version of our problem is NP-complete, which indicates NP-hardness of the SCSH problem.  $\square$

**Theorem 2.** The SCSH problem is APX-hard, unless  $P=NP$ .

*Proof.* Assume that SCSH admits a PTAS. Then for every  $\varepsilon > 0$  we can compute in polynomial time a triangle-connected  $(k', \mathcal{P})$ -truss community  $C'$  with  $|C'| = s, q \in C'$  and  $k' \geq (1-\varepsilon) \cdot k_{max}$ , where  $k_{max}$  is the maximum triangle-connected trussness among all feasible communities.

Consider the decision variant of SCSH: given an instance  $(H, \mathcal{P}, q, s, k)$ , decide whether there exists a community  $C$  with  $|C| = s, q \in C$  and  $\tau_\Delta(C) = k$ . And we have prove this decision problem is NP-complete.

We now use the PTAS to solve the decision problem in polynomial time. Set  $\varepsilon = \frac{1}{k+1}$ . Since  $k$  is a fixed constant,  $\varepsilon$  is

also a fixed constant. Hence, the PTAS runs in polynomial time.

With  $\varepsilon = \frac{1}{k+1}$  we have  $(1-\varepsilon)k = k - \frac{k}{k+1} = k - 1 + \frac{1}{k+1}$ , so  $k-1 < (1-\varepsilon)k < k$ . Because triangle-connected trussness is integral, the open interval  $(k-1, k)$  contains no integer. Thus:

- If  $k' \geq k$ , then  $k_{max} \geq k$  must hold (otherwise  $k' \leq k_{max} \leq k-1$ , contradiction).
- If  $k' \leq k-1$ , then  $k_{max} \leq k-1$  must hold (otherwise  $k_{max} \geq k \Rightarrow k' \geq (1-\varepsilon)k_{max} > k-1 \Rightarrow k' \geq k$ , contradiction).

Therefore the single comparison “ $k' \geq k$ ?” gives the correct YES/NO answer to the decision problem.

The whole procedure is polynomial, so we have solved an NP-complete problem in polynomial time  $\Rightarrow P=NP$ , a contradiction. Hence the assumed PTAS cannot exist, and SCSH is APX-hard unless  $P=NP$ .  $\square$

### III. STATE-OF-THE-ART FRAMEWORK

Size-bounded community search maximising cohesiveness has been explored extensively in homogeneous graphs under the  $k$ -core and  $k$ -truss models. We first retrace the methodological development of existing methods and then discuss the rationales for the algorithms proposed for our problem.

**SOTA framework [31], [34].** Interestingly, SOTAs use include-exclude backing tracking to systematically explore node sets that potentially lead to the desired result, as shown in Algorithm 1, which is decorated with upper/lower bound-based pruning, candidate reductions, and branching strategies. *Upper/lower bound-based bounding.* Whenever Algorithm 1 enters a new recursion state, it attempts to stop exploring non-promising recursion states by estimating an upper bound on  $(C, R)$ , i.e., the highest cohesiveness, and if the upper bound is no better than the lower bound ( $\bar{k}$ ), the current recursion state and its descendants cannot lead to a result with cohesiveness greater than  $\bar{k}$ . Notice that the upper bound is based on both  $C$  and  $R$ , which secures the correctness of the bounding (we will revisit the detail later). As such, Algorithm 1 only explores recursion states that cannot be bounded.

*Heuristic during search.* SOTAs propose corresponding scores for nodes in the candidate set  $R$ , which ensures that nodes in  $R$  contribute less to the cohesiveness of  $C$  as recursion goes deep (line 6). This enables the search to produce a result with high cohesiveness at early search stages, which enhances the upper/lower bound-based bounding.

*Eager candidate reductions.* SOTAs also incorporate candidate reductions based on the coreness (or trussness), distance, etc. The reduction is applied whenever Algorithm 1 enters a new recursion state (line 1).

*Branching strategy.* SOTAs propose dominance-based branching rules. The key idea is that, given two nodes  $u$  and  $v$ , if  $u$  dominates  $v$ , then a higher cohesiveness result must be induced by a recursion state including both  $u$  and  $v$  or a recursion state excluding  $v$ . This can be extended to when  $u$  dominates multiple nodes (lines 7 to 10).

**Opportunities.** Based on the above discussions, we identify the following opportunities, bearing SOTAs in mind.

---

**Algorithm 1: B&B**

---

**Input:** Partial community  $C$ , Candidates set  $R$   
**Output:** Optimal community  $C^*$   
/\* Initially  $C=\{q\}$ ,  $R$  contains nodes reachable from  $q$ ,  $\bar{k}$  is a heuristic result as a lower bound \*/  
1 Reduce  $(C, R)$  ;  
2 **if**  $|C|=s$  **then**  
3      $k \leftarrow$  cohesiveness of  $C$ ; // trussness or coreness  
4     **if**  $k > \bar{k}$  **then**  $\bar{k} \leftarrow k$ ,  $optC \leftarrow C$  ;  
5 **if**  $|C| < s$  &&  $R \neq \emptyset$  &&  $UB(C, R) > \bar{k}$  **then**  
6      $v \leftarrow$  pop a node with the highest score from  $R$  ;  
7      $\Phi \leftarrow$  compute node dominated by  $v$  ;  
8     **foreach**  $u \in \Phi$  **do**  
9          $X \leftarrow X \cup \{u\}$ ; //  $X$  is  $\emptyset$  initially  
10          $B\&B(C \cup \{v, u\}, R \setminus X)$  ;  
11      $B\&B(C \cup \{v\}, R \setminus X)$  ;  
12      $B\&B(C, R \setminus X)$  ;

---

First, although STOAs have made great efforts to prune unpromising recursions, they only explore homogeneous graph properties. We explore HIN properties, which lead to an effective heuristic algorithm that can quickly find a near-optimal result. This significantly enhances pruning effectiveness and reduces overall runtime.

Second, we introduce the  $k$ -combination recursion-based B&B framework (kcB&B) for solving the size constraint community search maximising cohesiveness. Compared to the include-exclude based framework runs in  $\mathcal{O}^*(2^n)$  regardless the size constraint  $s$ , kcB&B naturally runs in  $\mathcal{O}^*(C_n^{s-1})$ . Enhanced by other techniques proposed by us,  $\mathcal{O}^*(C_n^{s-1})$  can be further reduced to  $\mathcal{O}^*(n'^{s-2})$ , where  $n'$  can be much less than  $n$ .

Last but not least, compared to the  $k$ -core upper bound proposed in [31], SOTA for  $k$ -truss pays less attention to the upper bound while focusing more on the lower bound, leaving room for improvement.

In the next section, we propose our algorithms with novel optimisations based on the aforementioned opportunities.

#### IV. OUR METHODS

In this section, we introduce the lexicographic generation B&B framework first, which serves as the backbone of our proposed methods. Then, we discuss our main optimisations, including novel heuristics, upper bounds and total search order in great detail. Then, we concisely discuss and adapt several existing reduction rules to our framework.

##### A. Lexicographic generation framework

Due to the NP-hardness of our studied problem, no polynomial algorithm exists for solving it. As shown in SOTAs, we have to systematically try combinations of target nodes of size  $s$  and see if the target nodes induced  $\mathcal{P}$ -pairs (target nodes induced subgraph in  $G_P$ ) can form a triangle-connected  $(k, \mathcal{P})$ -truss with node size  $s$  while maximising the cohesiveness.

We find that the lexicographic generation [13] is the most efficient way to generate  $s$ -node sets from  $V(G_P)$ , especially when  $s$  is small, which nicely fits our application scenario. The reason is that lexicographic generation explores a generation tree with depth up to  $s$  to generate all  $s$ -node sets from

---

**Algorithm 2: The new framework for SCSH**

---

**Input:** HIN  $G$ , Meta-path  $\mathcal{P}$ , Query node  $q$ , Size constraint  $s$   
**Output:** Optimal community  $C^*$   
1  $C \leftarrow \{q\}$ ;  $C^* \leftarrow$  the best heuristic result; // Section IV-B  
2  $R \leftarrow$  target nodes reachable from  $q$  via  $\mathcal{P}$ -pairs;  
3 reduce  $R$  according to  $\tau_\Delta(C^*)$  and order the remaining  $R$ ; // Section IV-F  
4 **foreach**  $v \in R$  **do**  
5     // Section IV-E  
6     **if** stop conditions cannot be satisfied **then**  
7          $kcB\&B(C \cup \{v\}, R \setminus \{v\})$ ;  
8          $R \leftarrow R \setminus \{v\}$ ;  
9 **return**  $C^*$ ;  
9 **Function**  $kcB\&B(C, R)$  :  
10   **if**  $|C|=s$  **then**  
11     **if**  $\tau_\Delta(C) > \tau_\Delta(C^*)$  **then**  $C^* \leftarrow C$  ;  
12     // Section IV-C  
13     **if**  $|C| < s$  &&  $|R| \geq s - |C|$  &&  $UB(C, R) > \tau_\Delta(C^*)$  **then**  
14         // Section IV-D  
15          $v \leftarrow$  the most dominating node in  $R$ ;  
16          $\Phi \leftarrow$  nodes in  $R$  dominated by  $v$ ;  
17         **foreach**  $u \in R \setminus \Phi$  **do**  
18              $kcB\&B(C \cup \{u\}, R \setminus \{u\})$ ;  $R \leftarrow R \setminus \{u\}$ ;

---

$|V(G_P)|$ . In contrast, the inclusion-and-exclusion-based method explores a generation tree with depth up to  $|V(G_P)|$ . Due to the differences in maximum recursion depth, the lexicographic generation naturally exhibits better scalability.

To explain the algorithm, we first introduce a concept.

**Definition 5. Triangle connected trussness of a target node set:** Given a target node set  $C$ , the triangle connected trussness of  $C$  is the maximum  $k$  such that  $C$  induced subgraph of  $G_P$ , i.e.,  $G_P[C]$ , contains a triangle-connected  $(k, \mathcal{P})$ -truss including all nodes in  $C$ , denoted by  $\tau_\Delta(C)$ .

Below, we call ‘triangle connected trussness’ as ‘trussness’ for brevity. Besides, under the context of  $G_P$ , trussness in  $G_P$  is the same as trussness of  $\mathcal{P}$ -pairs.

Algorithm 2 shows our new search framework based on the lexicographic generation. The recursive function kcB&B serves as the backbone, systematically exploring  $s$ -node sets. Algorithm 2 first obtains a heuristic result that serves as an initial (trussness) lower bound (line 1). It then initialises the result set  $C$  by including the query node  $q$  and collects the complete candidates ( $R$ ), i.e., target nodes that are  $\mathcal{P}$ -pair reachable from  $q$ . It leaves room for ordering optimisations, which can apply novel techniques that can lead to potential early terminations (line 5). Then, kcB&B is called to explore  $s$  node sets from  $R$ , which only contains target nodes potentially leading to better results (line 3). When kcB&B enters a recursion state with  $(C, R)$  having  $|C|$  less than  $s$ , it attempts to stop and backtrack to other more promising recursion states by evaluating two bounding rules (line 12). The first size-based bounding ( $|R| \geq s - |C|$ ) is trivial, while the latter, i.e., deriving an upper bound based on both  $C$  and  $R$  and using it to compare to the best result up to the time, will be discussed later. When kcB&B reaches to  $s$ -node set (line 10), it derives  $\tau_\Delta(C)$  and updates  $C^*$  if necessary.

The completeness of Algorithm 2 is clear since it is based

on the lexicographic generation. Suppose all the reductions, bounding, branching rules and early terminations are correct (which we will discuss and prove in great detail),  $C^*$  is optimum when Algorithm 2 terminates. Ignoring optimisations to be discussed, Algorithm 2 runs in  $\mathcal{O}^*(\binom{V(G)}{s-1})$ .

**Edge sets generation (ESG) based approach.** The backbone of Algorithm 2 is to try node sets of size  $s$ , i.e., node sets generation (NSG), which is straightforward by the definition of our problem. However, as shown in Definition 5, the result is not a node-induced subgraph of  $G_P$ . This indicates an alternative to Algorithm 2, i.e., we try edge sets till the node size reaches  $s$ . In fact, this approach is more natural for truss-based cohesiveness since trussness is defined on edges, and the result is an edge-induced subgraph. We also attempt this approach and discover several interesting experimental results. Since this approach shares great similarities with Algorithm 2, we omit its details to avoid duplication.

Below, we start to discuss detailed optimisations. Unless explicitly discussed, optimisations apply to both Algorithm 2 (NSG) and ESG, with NSG being the default context.

### B. Lower bound initialization

One of the critical speedups for Algorithm 2 is to have a near-optimal result, i.e., a tight initial (trussness) lower bound. This is because a near-optimal result can be utilised to reduce the candidate set and bound the search. In this subsection, we utilise HINs to propose a polynomial algorithm that effectively initialises a tight lower bound.

**Rationale.** We can directly design heuristics and greedy algorithms to generate  $s$  node sets by including nodes with high trussness and see if we can get a quality result. The challenge is how we can evaluate the quality. As such, we first discuss the optimality of the lower bound for our problem. We then demonstrate that the quality lower bounds can be derived efficiently by leveraging the properties of HINs.

**Optimum lower bound.** The clique of  $G_P$  containing  $q$  provides a lower bound for the trussness of  $G_P$ , i.e., if there exists a  $k$ -clique containing  $q$  then there exists a triangle-connected truss containing  $q$  with trussness at least  $k$ . The optimum lower bound (denoted by  $\underline{k}$ ) by considering the size constraint is shown as follows.

$$\underline{k} = s \quad \text{if } \omega_q(G_P) \geq s, \quad (1)$$

where  $\omega_q(G_P)$  is the maximum clique size in  $G_P$  containing  $q$ .

Based on the above definition, to get a good lower bound, we want to find large cliques containing  $q$  in  $G_P$ . However, this could be hard, especially if we treat  $G_P$  as a homogeneous graph and ignore the fact that  $G_P$  is derived via instances of meta-path. To show why finding large-sized cliques in  $G_P$  is easy, we show an existing concept below.

**$\mathcal{P}$ -star [30].** Given an HIN  $H$  and a symmetric meta-path  $\mathcal{P} = (A_1, A_2, \dots, A_{l+1})$ , a set of nodes of type  $A_1$  is a  $\mathcal{P}$ -star if there is a node  $a_{\frac{l+1}{2}}$  of type  $A_{\frac{l+1}{2}}$  s.t. for each  $u$  in this set,  $a_{\frac{l+1}{2}}$  is a  $\frac{\mathcal{P}}{2}$ -neighbor of  $u$ , i.e., they are  $\frac{\mathcal{P}}{2}$  reachable.

**Observation 1.** Every  $\mathcal{P}$ -star is a clique in  $G_P$ .

### Algorithm 3: SCSHHeu

---

**Input:** HIN  $G$ , Meta-path  $\mathcal{P}$ , Query node  $q$ , Size constraint  $s$   
**Output:** The heuristic solution  $\tilde{C}$ , lower bound  $\tau_\Delta(\tilde{C})$

```

1  $S \leftarrow$  the set of seed cliques; //  $\mathcal{P}$ -stars containing  $q$ 
2 if  $|\arg \max_S \{|S| \mid S \in S\}| \geq s$  then
3    $\tilde{C} \leftarrow$  any  $s$  node set of  $S$  that contains  $q$ ;
4 else
5    $\tilde{C} \leftarrow \emptyset$ ;
6   sort  $S$  in non-increasing order according to  $|S|$  ( $S \in S$ );
   // We can terminate the loop after a few attempts
7   while  $S \neq \emptyset$  do
8      $C' \leftarrow S.\text{pop}()$ ;
9      $S' \leftarrow S \cup \mathcal{P}\text{-stars not containing } q \text{ but } \frac{\mathcal{P}}{2} \text{ reachable from } q$ ;
10    while  $S' \neq \emptyset$  do
11       $C'' \leftarrow \arg \max_S \{|E(G_P[S \cup C'])| \mid S \in S'\}$ ;  $S' \leftarrow S' \setminus \{C''\}$ ;  $C' \leftarrow C' \cup C''$ ;
12      if  $|C'| \geq s$  then
13        reduce  $C'$  to size  $s$  by progressively removing
        edges with the lowest trussness;
14        if  $q \in C'$  &&  $\tau_\Delta(\tilde{C}) < \tau_\Delta(C')$  then
15           $\tilde{C} \leftarrow C'$ 
16 return  $\tilde{C}$  and  $\tau_\Delta(\tilde{C})$ ;

```

---

Based on Observation 1, we can quickly get a set of cliques containing  $q$  by simply applying breadth first search on  $G$  (HIN) with type  $A_{\frac{l+1}{2}}$  nodes that are  $\frac{\mathcal{P}}{2}$  reachable from  $q$ . We treat these  $\mathcal{P}$ -stars as *seed cliques*.

The seed cliques could be as large as  $s$ , as shown in Equation 1, which leads to an optimum result directly, and Algorithm 2 does not need to invoke recursive calls. However, the seed cliques could be small. This motivates us to derive a tighter lower bound based on seed cliques.

**Clique-based lower bound.** We observe that if we union multiple seed cliques with more edges in their induced subgraph of  $G_P$ , the chance of forming high trussness subgraphs is high. Based on this rationale, the complete heuristic algorithm is shown in Algorithm 3.

If there is no seed clique ( $\mathcal{P}$ -star) containing  $q$  with size no less than  $s$ , Algorithm 3 initialises  $\tilde{C}$  as  $\emptyset$  since we do not have any feasible lower bound yet. Then, Algorithm 3 gives large seed cliques opportunities. For each large seed clique  $C'$  (line 8), it will be progressively enlarged by greedily merging other seed cliques  $C''$  that bring the most edges w.r.t.  $C'$  up to the time. After a union, if  $C'$  is larger than  $s$ , Algorithm 3 reduces it to fit the size and then replaces it with the best lower bound if necessary. Notice that Algorithm 3 is shown as giving each seed clique a chance; in fact, we can terminate it after a fixed number of rounds. Algorithm 3 returns  $\tilde{C}$  with  $\tau_\Delta(\tilde{C})$  as output.

**Computational cost.** The time complexity is  $\mathcal{O}(c \cdot |E(G_P[C'])|^{1.5})$ . The dominating parts are the two *while* loops together, having a time complexity of  $\mathcal{O}(|S| \cdot |S'| \cdot |E(G_P[C'])|^{1.5})$  (notice we can set a constant  $c$  and terminate this nested loop much earlier). The innermost loop is dominated by computing edge trussness and iteratively deleting edges, which takes  $\mathcal{O}(|E(G_P[C'])|^{1.5})$ .

**Example 1.** As shown in Fig. 1(a), consider  $q = a_6$ ,  $\mathcal{P} = (A, P, A)$ , and  $s = 7$ . Let  $S = \{p_2, p_3, p_4, p_7\}$ . We denote  $\mathcal{P}$ -stars using the centre nodes of the instances of  $\mathcal{P}$ . The largest  $\mathcal{P}$ -star is

$\{a_6, a_8, a_{10}, a_{11}\}$ , which is smaller than  $s$ . Therefore, Algorithm 3 proceeds to the “else” branch.

After sorting,  $\mathcal{S}=\{p_3, p_2, p_7, p_4\}$ . We first select  $p_3$  as the initial seed clique, i.e.,  $C'=\{a_4, a_5, a_6, a_7\}$  and update  $\mathcal{S}'=\{p_2, p_7, p_4, p_1, p_5, p_6\}$ . We then merge  $p_1$  into  $C'$  since  $p_1$  contributes the largest number of edges. As the resulting size is  $|C'|=7$ , which satisfies the size constraint, no size reduction is needed. This yields the final result  $\{a_1, a_2, a_3, a_4, a_5, a_6, a_7\}$  with trussness of 4, which is the best result returned by Algorithm 3.

### C. Bounding techniques

In this subsection, we present how to utilise a known result (the heuristic result), i.e., a lower bound, to bound the search tree by comparing it to an upper bound.

**Rationale.** Given a non-trivial recursion state with  $C$  and  $R$  ( $|C| < s$  and  $|R| \geq s - |C|$ ), we want to optimistically estimate the best trussness after moving arbitrary  $s - |C|$  nodes from  $R$  to  $C$ , i.e., an upper bound for this recursion state, denoted by  $UB(C, R)$ . Clearly, as long as  $UB(C, R)$  is admissible, that is, 1)  $UB(C, R)$  is no less than any results, in terms of trussness, induced by the current recursion state, and 2)  $UB(C, R)$  is no greater than the current lower bound, we can stop branching at the current recursion state. Algorithm 2 can explore (backtrack to) other recursion states that cannot be bounded.

As discussed, SOTAs all involve the idea above. In [31], the main idea is as follows. It is assumed that 1)  $s - |C|$  nodes from  $R$  that are mostly connected to  $C$  would be added to  $C$ , 2) these nodes all exhibit high coreness, and 3) all the edges incident from these nodes would contribute to nodes in  $C$  with the minimum coreness. The process of deriving the upper bound is dynamic, i.e., when all the nodes in  $C$  with minimum coreness have been increased by 1, the remaining edges will be added to the minimum coreness nodes until all incident edges are consumed. After that, the updated minimum coreness among all nodes in  $C$  serves as the upper bound.

On the other hand, STOA for truss-based model [34] is mainly based on the current trussness of  $C$  and the number of nodes in  $R$ . It is based on the fact that when moving a node from  $R$  to  $C$ , the trussness of  $C$  will be increased by at most 1. Then, given the trussness of  $C$ , [34] computes how many nodes are needed, and if  $R$  cannot provide sufficient nodes, the branch can be bounded. The admissibility of this bound lies in the fact that not every node can increase the trussness.

Clearly, the concept presented in [31] is more refined, as it considers necessary structural conditions based on the cohesive subgraph model. If it can be adapted to our problem, we could attain a much tighter bound compared to [34].

We shall present two non-trivial upper bounds—one for ESG and one for NSG. As mentioned, ESG is more natural when deriving the upper bound using the idea of [31]. Our contributions for designing ESG upper bounds are twofold: 1) to address the challenges induced by the cohesiveness difference, and 2) to address the inconsistency between edge inclusion and node inclusion (i.e., adding an edge may not increase the size of  $C$ ).

Then, we propose a novel upper bound for NSG, which is more challenging since the results of our model are edge-induced graphs, i.e., when NSG generates a node set  $C$ , it is not necessarily true that all edges in  $G_P[C]$  are in the result.

1) *Upper bounds for ESG:* We consider a non-trivial recursion state with  $C$  and  $R$ , where both  $C$  and  $R$  contain edges in  $G_P$ , and we use  $V(C)$  to denote the nodes contained in  $C$ .

**ESG upper bound computation.** The ESG upper bound is based on the accurate trussness for edges in  $C$  and on how  $R$  can increase the trussness of  $C$  optimistically.

The trussness for each edge in  $C$  can be progressively updated during the search process and can be treated as known information. We can partition edges according to their trussness. Our proposed upper bound starts with the minimum trussness edges in  $C$  denoted by  $E_{min}$  with trussness of  $\tau_{\Delta}(C)$ , where  $\tau_{\Delta}(C)$  is trussness of  $C$ .

To establish the optimistic while tight upper bounds, we assume edges in  $R$  which can contribute to  $R$  the most *capable triangles* (c-triangle), i.e., triangles consisting of edges with trussness no less than  $\tau_{\Delta}(C)$ , will be added to  $C$ . We also assume that all these capable triangles will contribute to  $E_{min}$  up to the time. Additionally, since including an edge may or may not introduce a new node, the optimistic number of c-triangles is derived as follows.

*Edges with no extra nodes.* For all these edges that form c-triangles with  $C$ , newly induced c-triangles are assumed to contribute to the trussness of  $E_{min}$  up to the time.

*Edges with extra nodes.* For edges with extra nodes, for each of such extra node  $u$ , we compute its score in terms of the number of c-triangles that  $u$  contributes to  $C$  (at least one of the nodes of such c-triangle is in  $V(C)$ ) in the  $C \cup R$  induced subgraph of  $G_P$ . For the top  $s - |V(C)|$  of the nodes with the highest score, they are assumed to move to  $C$  and contribute to the support of  $E_{min}$  up to the time.

For these optimistically estimated c-triangles assumed to be added to  $C$ , each edge in  $E_{min}$  will consume one of the c-triangles up to the time. When the support of every edge in  $E_{min}$  has been increased by 1, the estimated upper bound is increased by 1. Then  $E_{min}$  is updated by including edges with minimum trussness up to the time. We repeat this process until all these c-triangles have been consumed.

*Admissibility.* The above process naturally follows the trussness definition to optimistically estimate the trussness of enlarged  $C$ . In fact, 1) the number of c-triangles assumed to be added is an overestimation, 2) not all the c-triangles happen to contribute to the edge with the minimum trussness, and 3) after enlarging  $C$ ,  $C$  contains more edges while these edges are assumed to have high trussness.

*Computational cost.* The dominant cost of computing the upper bound is the number of triangles in  $C \cup R$  induced subgraph of  $G_P$  (edge induced subgraph), which takes  $\mathcal{O}(|C \cup R|^{1.5})$ .

**Grouped ESG upper bound computation.** Given two edge sets  $C$  and  $C'$  where  $C' \subseteq C$ , we have the fact that  $\min\{C'\}$  is no less than  $\min\{C\}$  in terms of edge trussness. This implies that the ESG upper bound computation can be applied to every subset of  $C$  (denoted by  $C'$ ) while the corresponding optimisti-

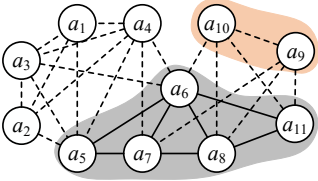


Fig. 3. An example of upper-bound

cally estimated c-triangles shall be constructed according to  $C'$  and  $R$ . Since each such  $C'$  leads to a feasible upper bound, by trying different  $C'$ , we may have a tighter lower bound. Especially in the case where  $C'$  contains low trussness edges while weakly connecting to  $R$ . Observed this, we follow [31] and apply edge upper bound for each subset of  $C^{\leq \tau} \in C$  s.t. the maximum trussness of edges in  $C^{\leq \tau} \in C$  is no greater than each  $\tau \in [\tau_{\min}(C), \tau_{\max}(C)]$ . The minimum upper bound among these serves as the grouped ESG upper bound.

**Computational cost.** It computes the edge upper bound at most  $|C|$  times, leading to a total complexity of  $O(|C| \cdot \text{ESG}())$ , where  $\text{ESG}()$  is the complexity of ESG upper bound computation.

**Potential of tightening the upper bound.** Since our search framework progressively enlarges  $C$  via the depth-first conversion, we pass down the current tightest upper bound and compare it to the edge-grouped upper bound and see if the upper bound can be further refined.

**ESG upper bound summary.** Given the fact that all the discussed upper bounds are feasible (including [34]), we always apply the tightest one.

**Example 2.** Refer to Fig. 3, the derived graph  $G_P$  for Fig. 1(a) when  $\mathcal{P}=(A, P, A)$ ,  $s=7, q=a_6$ . Let  $C=\{(a_5, a_6), (a_5, a_7), (a_6, a_7), (a_6, a_8), (a_6, a_{11}), (a_7, a_8), (a_8, a_{11})\}$  (highlighted in gray circle). The supports of  $(a_5, a_6)$ ,  $(a_5, a_7)$ ,  $(a_6, a_{11})$ ,  $(a_7, a_8)$ , and  $(a_8, a_{11})$  are 1, while that of  $(a_6, a_7)$  and  $(a_6, a_8)$  are 2 in  $G_P[V(C)]$ .  $R$  contains all edges not connecting to  $a_4$  and not in  $C$ .

We need to select  $s-|V(C)|=2$  nodes with the highest score, namely  $\{a_9, a_{10}\}$  (highlighted in orange circle). Because  $a_{10}$  forms 3 triangles with  $(a_6, a_8), (a_6, a_{11})$  and  $(a_8, a_{11})$ , and  $a_9$  forms 2 triangles with  $(a_7, a_8)$  and  $(a_8, a_{11})$ . After adding  $a_{10}$ , the updated support in  $G_P[V(C) \cup a_{10}]$  are:  $(a_5, a_6)$  and  $(a_5, a_7)$  remain 1, and  $(a_6, a_7), (a_6, a_8), (a_6, a_{11}), (a_7, a_8)$ , and  $(a_8, a_{11})$  become 2. After further adding  $a_9$ , the supports of all edges are 2. Therefore, the ESG upper bound is  $2+2=4$ .

For the grouped ESG upper bound, the grouped edges in  $C$  are:  $C^{\leq 1}=\{(a_5, a_6), (a_5, a_7), (a_6, a_{11}), (a_7, a_8), (a_8, a_{11})\}$ ,  $C^{\leq 2}=C$ . For  $C^{\leq 1}$  and  $C^{\leq 2}$ , the ESG upper bound is 3 and 4, respectively. So, the grouped ESG upper bound is  $\min\{3, 4\}=3$ .

2) **Upper bound for NSG:** Unfortunately, our proposed ESG upper bound does not work for NSG. This is because these upper bounds are estimated by increasing the minimum edge trussness. In ESG, these bounds work since once an edge is included in  $C$ , it cannot be removed from  $C$ . This is untrue when  $C$  is a set of nodes because edges with low trussness could be removed, and only nodes added to  $C$  cannot be removed. Therefore, we shall devise upper bounds by exploring other properties.

**Rationale.** Reminder that according to Definition 3, the trussness of a node  $u$  for  $C$  is the maximum edge trussness for

edges incident to  $u$  in  $G_P[C]$ . As such, to estimate an upper bound for NSG, we shall assume that triangles are added to edges incident to the node with the maximum trussness, which is totally different from edge-based upper bounds.

Besides, to secure the optimistic estimation, we shall consume the least number of triangles so that the trussness of a node can be increased. Luckily, one of the properties of the truss model provides us with such an opportunity.

**Property 1.** For a  $(k, \mathcal{P})$ -truss, there must be at least  $\frac{k(k-1)}{2}$  edges with  $\tau(u, v) \geq k$  in  $G_P$ .

Based on the above property, in order to increase the trussness of a subgraph with trussness of  $k$  optimistically, we just need to ensure that there are  $\frac{(k+1)k}{2}$  edges that are with trussness of  $k+1$  after adding some triangles.

We are ready to show the upper bound for NSG given a non-trivial recursion state with  $C$  and  $R$ , where both  $C$  and  $R$  contain nodes only.

**NSG upper bound.** The upper bound is based on the accurate  $\tau_{\Delta}(C)$  in  $G_P[C]$ , and how  $R$  can maximally increase  $\tau_{\Delta}(C)$ .

**Capable triangle and edge of NSG.** A triangle is capable if it is 1) formed by edges with trussness no less than  $\tau_{\Delta}(C)$ , and 2) triangle reachable from  $C$  via triangles satisfying condition 1). A capable edge is defined similarly to the capable triangle.

**Optimistic number of capable triangles.** For each node  $u \in R$ , let its score be the number of capable triangles that  $u$  is involved in  $G_P[C \cup R]$ . For the top  $s-|C|$  nodes with the highest score in  $R$ , they are assumed to move to  $C$ , and the sum of their scores is the optimistic number of triangles that contribute to nodes in  $C$ , denoted by  $|\Delta_{opt}|$ .

**Optimistic number of edges.** Additionally, we also need the top  $s-|C|$  nodes in  $R$  that can potentially contribute the highest number of capable edges, denoted by  $|E_{opt}|$ . We need  $|E_{opt}|$  because we assume that edges incident to the  $s-|C|$  nodes exhibit sufficiently high trussness. They may directly contribute to nodes in  $C$  with the minimum trussness.

**Unifying optimistic triangles and edges.** To ensure an optimistic result, we assume that there exists a size  $s-|C|$  node set that is optimistic in terms of both capable edges and triangles.

**How triangles and edges are optimistically consumed.** Based on the above discussion, given  $C$  with trussness of  $k=\tau_{\Delta}(C)$ , it is not in  $k+1$  truss since it has less than  $\frac{k(k+1)}{2}$  edges with trussness of  $k+1$  shown in Property 1. So initially, the upper bound is the same as  $\tau_{\Delta}(C)$ , i.e.,  $k$ . Clearly, after adding the discussed  $s-|C|$  nodes, it brings  $|E_{opt}|$  edges with sufficiently large trussness. As such, we need at least  $\frac{k(k+1)}{2} - |E_{opt}|$  (assuming  $|E_{opt}|$  is smaller than  $\frac{k(k+1)}{2}$ ). Otherwise, we first increase  $k$  till  $\frac{k(k+1)}{2} > |E_{opt}|$  edges with trussness of  $k+1$  to increase the trussness of  $C$  by 1. To do so, we assume that triangles in  $\Delta_{opt}$  are used to increase the trussness of edges in  $G_P[C \cup R]$  whose trussness is closest to  $k+1$ . Every time the trussness of an edge is increased, one triangle in  $\Delta_{opt}$  is consumed. If all triangles in  $\Delta_{opt}$  have been consumed and there are still no  $\frac{k(k+1)}{2} - |E_{opt}|$  edges with trussness of  $k+1$ , the upper bound does not change. If there are triangles in  $\Delta_{opt}$  left, we set  $k=k+1$  and repeat the above process till all triangles are consumed. The updated upper bound serves as our NSG upper bound.

**Admissibility.** The admissibility is clear. First, the condition that we increase the trussness of  $C$  is the minimum requirement to form a  $k+1$ -truss. Then, we assume  $s-|C|$  nodes bring the most number of capable triangles and edges to  $C$ . Third, we assume that all the capable triangles contribute to edges that are most likely to increase the trussness, and all the capable edges have sufficient trussness.

Due to the fact that we increase the trussness of edges with the maximum trussness,  $\tau_{max}(C') \geq \tau_{max}(C)$  cannot hold anymore for  $C' \subseteq C$ . As such, we cannot further tighten the upper bound as the one for ESG.

**Computational cost.** The time complexity of NSG upper bound is also dominated by triangles in  $G_P[C \cup R]$ , i.e.,  $\mathcal{O}(|E(G_P[C \cup R])|^{1.5})$ .

**NSG upper bound summary.** We can apply the minimum of the upper bound used in [34] and our proposed upper bound to the algorithm. If the derived upper bound is no greater than  $\tau_{\Delta}(C^*)$  (the lower bound up to the time), Algorithm 2 can backtrack to other recursion states.

**Example 3.** Using Fig. 3 as an example, the query setting and recursion state are the same as in Example 2, but change the edges set to the nodes set. The trussness of all nodes in  $C$  is 3, the two nodes with the highest scores are  $a_{10}$  and  $a_9$ , i.e.,  $|\Delta_{opt}|=5$  and  $|E_{opt}|=7$ . To increase the trussness of  $C$  to 4, we need  $\frac{4 \times 3}{2} - 3 = 3$  edges with trussness of at least 4, which consumes 3 triangles. After this step, the trussness of the edges become: 4, 4, 4, 3, 3, 3, 3. In the next iteration, to increase the trussness to 5, it would require  $\frac{5 \times 4}{2} - (3+3+1) = 10 - 7 = 3$  edges with trussness of at least 5. Because there are 3 edges with trussness of 4 after the previous iteration, it still requires 3 triangles. However, only 2 triangles remain, which is insufficient to support this increase. Therefore, the NSG upper bound is 4.

## D. Branching strategy

We present a novel branching strategy in this subsection.

**Rationale and SOTA.** Given a recursion state  $(C, R)$ , and two nodes  $u$  and  $v$  in  $R$ , if the cohesiveness of every result led by branching at  $v$  with  $R \setminus \{u\}$  is no more than that of the result by replacing  $u$  to  $v$ , then we can avoid branching at  $v$  in the current recursion state while preserving the optimum results.

For core and truss-based models, we can examine whether  $N(v) \subseteq N(u)$  holds in  $G[C \cup R]$  to see if  $v$  can be avoided or not. This is referred to  $u$  dominating  $v$ . In this paper, we call such dominance as  $u$  edge-dominating  $v$ , denoted by  $u \succeq_E v$ .

Based on the above discussion, within their framework, when branching at a node  $u$ , every node  $v$  dominated by  $u$  will not be branched, and branches including each pair of  $\{u, v\}$  will be attempted, in addition to the normal inclusion and exclusion branches.

We advance existing ideas in twofold. First, we propose a triangle-based dominance, which can be used to expand the node set that can be dominated by a node. Second, we propose a novel branching strategy that aims to minimise the number of branches as much as possible for each recursion state.

**Triangle based dominance.** Given a recursion state  $(C, R)$ , let  $\Delta(u)$  and  $\Delta(v)$  denote the set of triangles that  $u$  and  $v$  involve in  $G[C \cup R]$ , if  $\Delta(v) \subseteq \Delta(u)$ ,  $u$  is triangle-dominating  $v$  in  $G[C \cup R]$  denoted by  $u \succeq_{\Delta} v$ .

The above discussion is also correct for  $G_P$ .

**Combining edge and triangle dominance.** It is trivial to see that triangle dominance has the same effect as edge dominance. However, since the conditions of triangle dominance are stricter, we must apply it carefully. For a node  $u \in R$ , we first generate a node set that can be dominated by  $u$  using edge-dominance, denoted by  $\Phi_E(u)$ . Then, for nodes that cannot be edge-dominated by  $u$  but have common neighbours with  $u$ , we apply triangle-dominance to generate a node set that can be dominated by  $u$ , denoted by  $\Phi_{\Delta}(u)$ . Then  $\Phi(u) = \Phi_E(u) \cup \Phi_{\Delta}(u)$  is set of nodes that can be dominated by  $u$ .

**Effectiveness.** It is easy to see  $\Phi(u) \supseteq \Phi_E(u)$  always holds. This indicates that, using our proposed dominance, each node can potentially branch at fewer nodes in  $R$ .

Now, we are ready to propose the novel branch strategy.

**Rule 1.** Given a recursion state with  $C$  and  $R$ , select a node with the maximum dominance in  $R$ , i.e.,  $u$  is  $\arg \max_v \{|\Phi(v)| | v \in R\}$ . Then, for this recursion state, we only branch at each node  $u'$  in  $R \setminus \Phi(u)$ , i.e., each induced subproblem has an input of  $C \cup \{u'\}$  and  $R$ .

Rule 1 is a branching rule since it does not remove the dominated nodes from  $R$  for the subproblems induced by the current recursion state. Its correctness is clear based on the discussed dominance.

**Computational cost.** It is evident that the time complexity of computing triangle-based dominance is greater than that of edge-based dominance. Identifying all nodes involved in triangles requires  $\mathcal{O}(|E(G_P[C \cup R])|^{1.5})$  time, which dominates the overall cost.

**Example 4.** In Fig. 2,  $a_3$  dominates  $\{a_1, a_2\}$ ,  $a_4$  dominates  $\{a_1, a_2, a_3, a_5\}$ , and  $a_{10}$  dominates  $a_{11}$ . Therefore,  $a_4$  is chosen to reduce the number of branches. Since all these dominances are triangle-based, this example does not clearly highlight the effectiveness of triangle-based dominance over edge-based dominance. But we can consider a simple modification: change the edge  $(a_9, a_7)$  to  $(a_9, a_5)$ . After this change,  $a_{10}$  cannot dominate  $a_9$  based on edges. However,  $a_9$  can still be triangle-dominated, because  $(a_9, a_5)$  does not participate in any triangle. As a result,  $\Delta(a_9) \subseteq \Delta(a_{10})$ , and  $a_9$  can be triangle-dominated by  $a_{10}$ . This can expand the scope of domination of  $a_{10}$ , illustrating the effectiveness of triangle-based dominance.

Triangle-based dominance can be applied to ESG. Due to the similarity, we do not repeat it.

## E. A total search order and early termination

Thanks to the new framework, we can enforce a total search order for our algorithm, which can improve time complexity as well as bring early termination opportunities. This idea is popular and critical for many other problems, such as maximal clique and biclique enumeration. We now introduce it to the size-bounded community search problem.

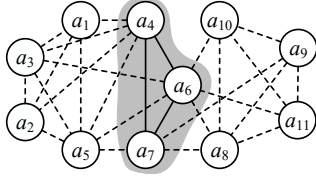


Fig. 4. An example of candidate reduction

**Our proposed ordering  $\mathcal{O}$ .** We propose a distance-first, truss-second-order. I.e., for  $R$  that cannot be pruned, we first sort nodes in  $R$  in non-decreasing order in terms of their distance to the query node  $q$ . Then, for the nodes with the same distance, nodes are sorted in non-increasing order in terms of their trussness in  $G_P[C \cup R]$ .

It is trivial to see that using  $\mathcal{O}$  as well as our introduced new framework, Algorithm 2 can terminate much earlier if one of the following conditions is satisfied in order.

**Stop Condition 1.** For the current  $v$ , if  $\text{dist}(v, q) > 1$ , Algorithm 2 can terminate.

**Stop Condition 2.** For the current  $v$ , if  $\tau_\Delta(v)$  in  $G_P[C \cup R]$  is no greater than  $\tau_\Delta(C^*)$ , Algorithm 2 can terminate.

**Stop Condition 3.** If the condition in our bounding technique is satisfied, Algorithm 2 can terminate.

The correctness of the above conditions is clear. Later, when analysing the time complexity, Stop condition 1 is critical for reducing the time complexity.

**Example 5.** Using Fig. 2 as an example, let  $q = a_6$  and  $\tau_\Delta(C^*) = 4$ . When  $R = \{a_2, a_9\}$ , Algorithm 2 terminates successfully, i.e. Stop Condition 1, because  $C \cup R$  is no longer connected to the query node  $q$ . Furthermore, when  $R = \{a_7, a_8, a_9, a_{10}, a_{11}\}$ , Algorithm 2 triggers Stop Condition 2 since there are no nodes with trussness of at least 5. Since the example of Stop Condition 3 closely resembles the proposed bounding techniques, we omit it here for brevity.

The above conditions can be refined slightly for ESG.

### F. Candidate reduction

In this subsection, we discuss several reduction rules that can be applied to a candidate set  $R$ . Notice that this subsection is mainly for self-completeness purposes since, except for the reduction rule involving our proposed lower and upper bounds, the distance-based reduction rules are based on properties of trussness and have been used in [29].

**Rule 2.** Given a search state  $(C, R)$ , for any node  $v \in R$ , if  $\tau_C(v) \leq \tau_\Delta(C^*)$  holds in  $G_P[C \cup R]$ ,  $v$  can be removed from  $R$ .

**Rule 3.** Given a search state  $(C, R)$ , for any node  $v \in R$ , if  $v$  is not  $(\tau_\Delta(C^*) + 1)$ -triangle reachable from  $C$  in  $G_P[C \cup R]$ ,  $v$  can be removed from  $R$ .

**Rule 4.** Given a search state  $(C, R)$  with size constraint  $s$ , for any node  $v$  in  $R$ , define  $\varepsilon = \max_{u \in C} \text{dist}_{G_P[C \cup R]}(u, v)$ . If  $\lceil \frac{\varepsilon(\tau_\Delta(C^*) + 2)}{2} \rceil > s$  then node  $v$  can be removed from  $R$ .

All the above rules can be trivially refined for ESG; therefore, we omit the details.

**Example 6.** Using Fig. 4 as an example. Let  $s = 4$ ,  $C = \{a_4, a_6, a_7\}$  (highlighted in the gray circle),  $R = \{a_1, a_2, a_3, a_5, a_8, a_9, a_{10}, a_{11}\}$  and  $\tau_\Delta(C^*) = 3$ . By Rule 2, no nodes can be pruned, because all nodes have trussness of at least 4. Additionally, by Rule 3,

nodes  $\{a_8, a_9, a_{10}, a_{11}\}$  are excluded from  $R$ , because they are not 4-triangle-connected with  $C$ . According to Rule 4, for  $a_1$ , It has  $\varepsilon = 2$  and  $\lceil \frac{2 \times (3 + 2)}{2} \rceil = 5 > 4$  thus,  $a_1$  can be discarded from  $R$ . The same applies to  $a_2$  and  $a_3$ . As such, the updated state becomes  $C = \{a_4, a_6, a_7\}$  and  $R = \{a_5\}$ .

### G. Wrap-up

We wrap up with the time complexity analysis.

**NSG.** The time complexity is  $\mathcal{O}(|N(q, G_P)| \binom{|V(G)|}{s-2} |E(G_P)|^{1.5})$ . Computing  $G_P$  can be bounded by  $\mathcal{O}(|V(G)||E(G)|)$ , where  $G$  is the input HIN.  $G_P$  can be computed much fast according to the query meta-path. The main cost of each recursion is induced by computing triangles in  $G_P[C \cup R]$ , which can be bounded by  $|E(G_P)|^{1.5}$  and should be much faster since we do not compute it from scratch. Compared to the complexity mentioned in Section IV-A, the exponential part is reduced to  $|N(q, G_P)| \binom{|V(G)|}{s-2}$  thanks to the total ordering and early Stop Condition 1, i.e., line 5 in Algorithm 2 is bounded by  $|N(q, G_P)|$  and Algorithm 2 only needs to check up to  $\binom{|V(G)|}{s-2}$  node sets.

**ESG.** The time complexity is  $\mathcal{O}(|N(q, G_P)| \binom{|E(G)|}{(s-1)(s-2)} |E(G_P)|^{1.5})$ . It has a much higher exponential part compared to NSG. This is because we have to add  $\frac{(s-1)(s-2)}{2}$  edges so that the node size can reach  $s-1$  in the worst case.

And Tab. II summarizes the relative tightness of all our bounds. The grouped ESG upper bound is tighter than the ESG upper bound. Although ESG explores more pruning opportunities, NSG exhibits much lower time complexity. We conduct comprehensive experiments to evaluate their practical performance.

TABLE II  
TIGHTNESS OF UPPER BOUNDS

| Upper Bound    | Tightness                    | Time Complexity                         |
|----------------|------------------------------|-----------------------------------------|
| ESG UB         | Relatively Loose             | $\mathcal{O}( C \cup R ^{1.5})$         |
| Grouped ESG UB | Tighter than ESG UB          | $\mathcal{O}( C  \cdot \text{ESG}())$   |
| NSG UB         | Comparable to Grouped ESG UB | $\mathcal{O}( E(G_P[C \cup R]) ^{1.5})$ |

## V. EXPERIMENTS

In this section, we present experimental results to demonstrate the effectiveness and efficiency of the proposed methods.

**Datasets.** We use 5 real datasets. Their statistics are shown in Table III, where  $|E(G_P)|$  represents the average edge count of the graphs  $G_P$  derived from different meta-paths  $\mathcal{P}$  in the meta-path pool.

TABLE III  
DATASETS

| Datasets    | Nodes      | Edges      | $ E(G_P) $    | $ \mathcal{A} $ | $ \mathcal{R} $ | $\#\mathcal{P}$ |
|-------------|------------|------------|---------------|-----------------|-----------------|-----------------|
| Amazon      | 13,136     | 209,746    | 5,929,707     | 5               | 4               | 10              |
| DBLP        | 37,791     | 170,794    | 4,501,017     | 4               | 4               | 10              |
| DoubanMovie | 37,595     | 1,713,902  | 26,234,499    | 6               | 6               | 10              |
| Aminer      | 439,504    | 1,130,039  | 354,466,972   | 6               | 6               | 10              |
| Freebase    | 12,116,680 | 35,231,642 | 1,428,764,912 | 8               | 36              | 100             |

**Algorithms.** We compare the following algorithms.

- SC-BRB [31]: Size-constrained  $k^*$ -core.
- ST-Exa [34]: Size-constrained  $k^*$ -truss.
- SCSHEP+: ESG equipped with all techniques.
- SCSHEV+: NSG equipped with all techniques.

Due to the fact that SC-BRB and ST-Exa are designed for homogeneous graphs, we adapt them with  $G_P$  as input.

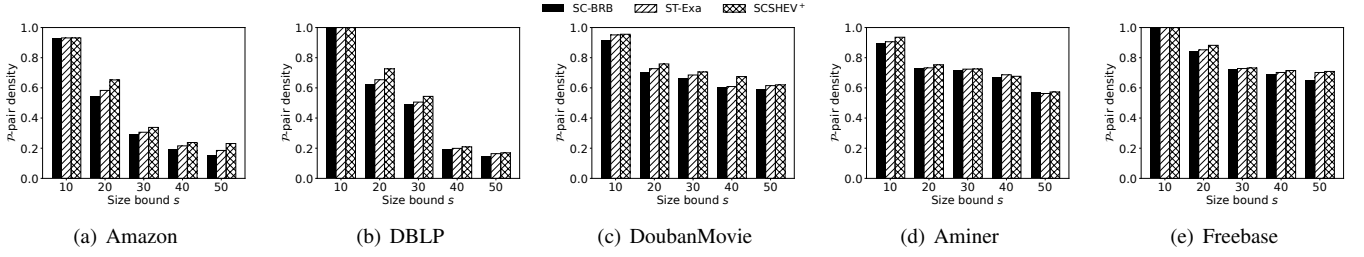


Fig. 5.  $\mathcal{P}$ -pair density

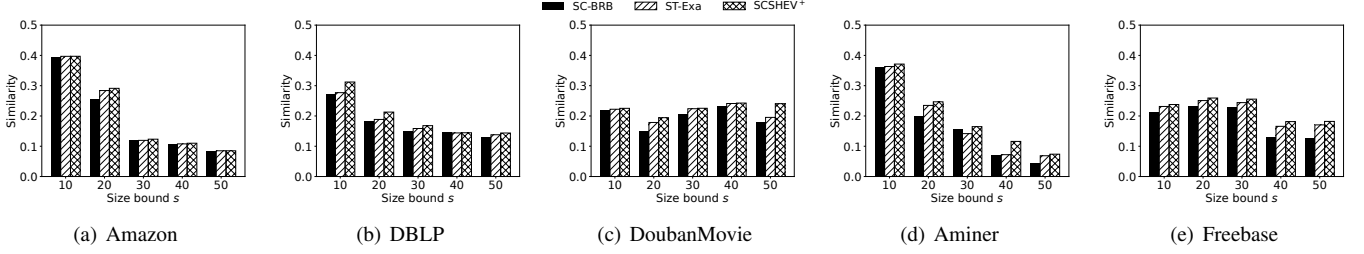


Fig. 6. Similarity

**Queries.** For each dataset, we construct a meta-path pool, with sizes reported in Table III. The maximum meta-path length is 6, following established settings in prior studies [9] and [30]. The size bound  $s$  varies from 10 to 50 in increments of 10, i.e.,  $s \in \{10, 20, 30, 40, 50\}$ , which is consistent with previous works [31], [34], and is chosen because larger values rarely yield additional valid or semantically meaningful triangle-connected  $(k, \mathcal{P})$ -trusses in HINs, due to triangle-connectivity constraints.

We generate 100 queries per dataset. For each query, a meta-path is randomly chosen in the pool, and 10 query nodes are sampled accordingly. Each query is tested under size bounds from 10 to 50, with a one-hour time limit. All algorithms are implemented in Python and evaluated on the same queries for fairness, running on Windows with an Intel(R) Xeon(R) W-2133 CPU (3.60GHz) and 32 GB RAM.

#### A. Effectiveness Evaluation

We evaluate the effectiveness of SCSHEV<sup>+</sup>, SC-BRB, and ST-Exa w.r.t., focusing on two widely used metrics: density and similarity.

**Density of  $\mathcal{P}$ -pairs.** The calculation of  $\mathcal{P}$ -pair density [9] is the same as the edge density in an homogeneous graph. Fig. 5 presents the density results. As the size bound  $s$  increases, density generally decreases. Among the compared methods, SCSHEV<sup>+</sup> consistently achieves the highest density, due to its stricter structural constraints. Notably, on the DBLP and Freebase datasets, it achieves a density of 1 when  $s=10$ , indicating that it discovers cliques under small size bounds.

**Similarity.** We use PathSim [24] to compute node similarity, and define the similarity of a community as the average PathSim score across all node pairs within a community. Fig. 6 shows that average similarity decreases as size bound  $s$  grows. SCSHEV<sup>+</sup> achieves the highest scores due to its denser, more semantically coherent communities.

**Case study.** In the case study on the Amazon dataset with query node i385,  $\mathcal{P}=(item, view, item)$ , and size bound 15. As shown in Fig. 7(a), ST-Exa returns a 6-truss without triangle connectivity,  $\mathcal{P}$ -pair density 0.533. Fig. 7(b) shows the result

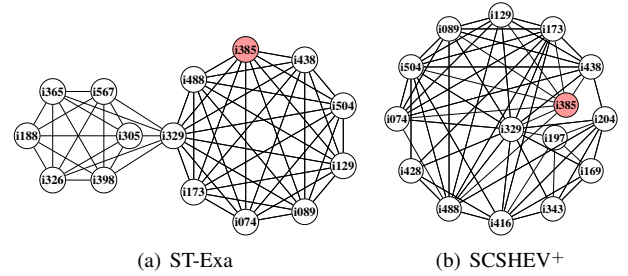


Fig. 7. Case study

returned by SCSHEV<sup>+</sup>, which is a triangle-connected 6-truss, with a higher  $\mathcal{P}$ -pair density of 0.562.

In HINs, a meta-path can capture meaningful indirect relationships between target nodes. Here,  $\mathcal{P} = (Item, View, Item)$  links two items viewed by users, reflecting co-view-based semantics, which is one of the most popular interactions useful for recommendations. Doing so, in the derived subgraph, items in a cohesive subgraph containing i386 exhibit high similarity with i386. In our model, the most cohesive  $k$ -truss requirement secures this, and the triangle connectivity further strengthens this. Therefore, our proposed model ensures items in the results are highly similar to i386 from the co-view-based semantics, which can serve as potential recommended items for users interested in (or have purchased) i386. The only problem left is if there are too many items to recommend, users can be confused or overwhelmed. The size constraint in our problem setup well mitigates this. Our model can recommend  $s$  items most related to i386 (i.e., most cohesive in terms of triangle connected truss).

Now let us go back to Fig. 7(a), ST-Exa returns two loosely connected sub item sets, suggesting mixed user interests and limited relevance to the query item i385. In contrast, Fig. 7(b) shows a single, tightly knit item set centred at i385.

#### B. Efficiency Evaluation

SC-BRB is not applicable to the SCSH problem. We slightly modify ST-Exa to make it compatible with our setting. Specifically, when  $|C|=s$ , the computation of trussness is replaced by triangle-connected trussness.

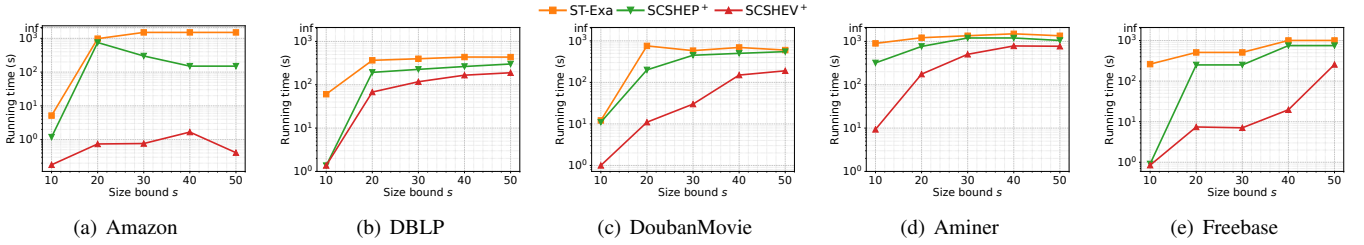


Fig. 8. Overall running times

**Against baseline algorithms.** The overall running time is presented in Fig. 8. As the size bound  $s$  increases, the running time also increases. This is because a larger  $s$  typically leads to a smaller  $k$ , which in turn reduces the number of pruned branches during the search. However, the running time does not always increase monotonically with  $s$ . For example, on the Amazon dataset, the running time of SCSHEV+ decreases significantly when  $s$  increases from 40 to 50. This is because the heuristic is able to identify a better lower bound when  $s=50$ , which greatly enhances the effectiveness of pruning and reduction. In contrast, when  $s=40$ , the heuristic fails to find such a bound, resulting in a larger search space and longer running time despite the smaller size bound.

Among all datasets, SCSHEV+ consistently achieves the best running time. As discussed in Section IV-G, although the ESG upper bound could be tighter than the NSG one, SCSHEP+ performs worse than SCSHEV+ in practice due to its high time complexity. Compared with ST-Exa, the SCSHEV+ achieves better efficiency for two main reasons. First, our heuristic does not require explicitly constructing  $G_P$  and runs in polynomial time, whereas ST-Exa involves exponential complexity, although both heuristic are clique-based. Second, our method is built upon the triangle-connected  $(k, \mathcal{P})$ -truss, while ST-Exa is based on the  $k$ -truss, which results in a larger search space.

TABLE IV  
AVERAGE RUNNING TIME(S) OF PREPROCESSING

| Dataset | Construct $G_P$ | Triangle Init. | Truss dec. | Enumeration | Overall  |
|---------|-----------------|----------------|------------|-------------|----------|
| Amazon  | 0.0141          | 0.0016         | 0.0094     | 0.3006      | 0.4526   |
| Aminer  | 1.6899          | 2.7899         | 19.0643    | 487.4445    | 510.9886 |

We divide the runtime into preprocessing and enumeration phases, where preprocessing includes building  $G_P$ , truss decomposition, etc. The times for these preprocessing steps are reported in Tab. IV with  $s=30$ , as preprocessing time remains consistent across different size constraints. It is evident that enumeration dominates the overall runtime.

**Different techniques.** To evaluate the effectiveness of different techniques, we implement four variants: SCSHEV+/H without heuristic lower bound, SCSHEV+/R without reduction rules, SCSHEV+/P without pruning rules based on upper bound, and SCSHEV+/B without branching rules. Fig. 9 shows their running times on the Amazon and Aminer datasets as  $s$  varies. We observe that running time increases with  $s$ , not only due to a larger search space but also because higher  $s$  tend to lower the trussness of the final result, reducing the effectiveness of both reduction and pruning. Among all algorithms, SCSHEV+ consistently achieves the best performance. Moreover, removing any of the techniques leads

to increased running times, confirming their contributions. Without reduction rules, the impact is the most pronounced, resulting in the largest increase in running time.

From the figure, we observe that the effectiveness of the techniques, in descending order of effectiveness, is: pruning, reduction, heuristic lower bounds, and branching rules. This is because all these techniques aim to shrink the search space, with pruning having the most pronounced impact.

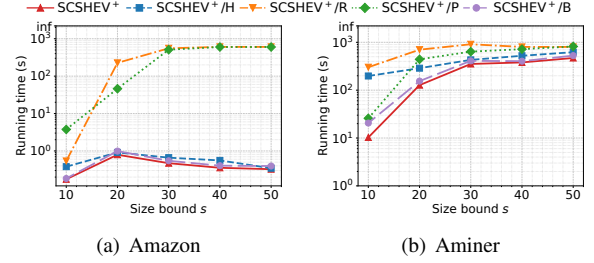


Fig. 9. Evaluate different techniques

Furthermore, Tab. V reports the gap between the heuristic lower bound and the optimal solution. Although the gap varies across datasets and parameters, it remains moderate in most cases, indicating that the heuristic is generally effective.

TABLE V  
GAP(%) OF  $\tau_{\Delta}$  BETWEEN HEURISTIC AND OPTIMAL SOLUTIONS

| Dataset | $s=10$ | $s=20$ | $s=30$ | $s=40$ | $s=50$ |
|---------|--------|--------|--------|--------|--------|
| Amazon  | 1.67%  | 15.00% | 22.14% | 15.00% | 33.33% |
| Aminer  | 10.51% | 13.19% | 15.53  | 19.65% | 18.75% |

**Different reduction rules.** We also evaluate the impact of individual reduction rules on running time. Here,  $R_1$  -  $R_4$  correspond to Rule 2-Rule 4. We incrementally incorporate these rules into SCSHEV, for example, SCSHEV+ $R_2$  includes both  $R_1$  and  $R_2$ , and so on. The running times on the Amazon and Aminer datasets are shown in Fig. 10. From the figure, we observe that the algorithm with all reduction rules, SCSHEV+ $R_3$ , achieves the best performance. Among Rules 2-4, Rule 3 is the most effective, as triangle connectivity eliminates the largest portion of candidate vertices.

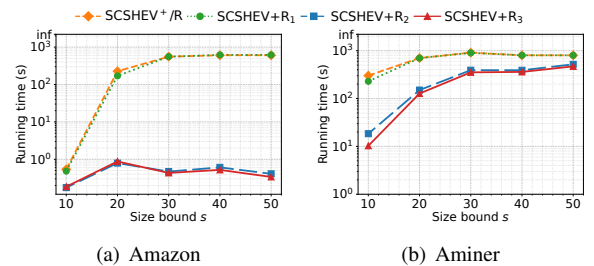


Fig. 10. Evaluate different reduction rules

**Impacts of upper bounds.** Fig. 11 illustrates the impact of upper bounds. Following the idea from [31], we define three

upper bounds:  $U_1 = \min_{v \in C} (\tau_{G_P[C \cup R]}(v))$ ,  $U_2 = \min_{v \in C} (\tau_{G_P[C]}(v)) + s - |C|$ , and  $U_3$ , which is our proposed NSG upper bound. Based on these, we introduce three pruning strategies:  $P_1$  corresponds to  $UB(C, R) = U_1$ ,  $P_2$  to  $\min\{U_1, U_2\}$ , and  $P_3$  to  $\min\{U_1, U_2, U_3\}$ . As shown in the figure, SCSHEV+ $P_3$  achieves the best performance, demonstrating the effectiveness of the pruning rules. On the Amazon dataset,  $P_2$  and  $P_3$  yield similar results, while on the Aminer dataset,  $P_1$  and  $P_2$  perform comparably. Furthermore, Tab. VI reports the number of explored branches. Thus, the number of pruned branches can be obtained by subtracting consecutive settings and shows that stronger upper bounds consistently eliminate more branches.

This suggests that, although these rules exhibit overlapping effects, they are not redundant. The degree of overlap may vary depending on the topology of the dataset. This can also be explained from the perspective of their underlying principles, as the three rules are derived from different structural viewpoints, which further explains why they are not redundant.

TABLE VI  
THE AVERAGE NUMBER OF EXPLORED BRANCHES

| Dataset | Algorithm     | s=10  | s=20    | s=30     | s=40     | s=50     |
|---------|---------------|-------|---------|----------|----------|----------|
| Amazon  | SCSHEV+ $P$   | 626.0 | 15090.3 | 658515.5 | 658332.3 | 658554.1 |
|         | SCSHEV+ $P_1$ | 561.9 | 14619.9 | 658512.0 | 658291.0 | 658189.9 |
|         | SCSHEV+ $P_2$ | 293.3 | 199.8   | 582.0    | 582.0    | 582.0    |
|         | SCSHEV+ $P_3$ | 35.3  | 88.9    | 69.5     | 173.4    | 112.3    |
| Aminer  | SCSHEV+ $P$   | 316.2 | 11721.8 | 24044.4  | 36412.7  | 54411.9  |
|         | SCSHEV+ $P_1$ | 178.6 | 1935.3  | 18129.5  | 23609.6  | 51419.1  |
|         | SCSHEV+ $P_2$ | 106.9 | 546.6   | 13299.4  | 19635.7  | 31534.6  |
|         | SCSHEV+ $P_3$ | 96.5  | 337.3   | 10639.5  | 15708.6  | 25227.7  |

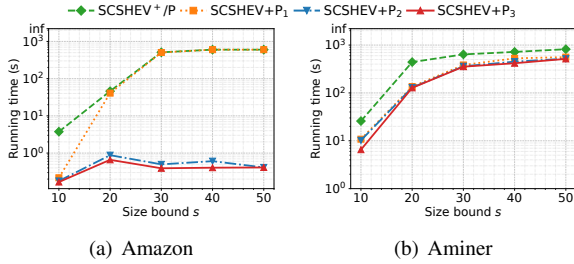


Fig. 11. Evaluate different pruning rules

**Scalability.** In this experiment, we evaluate the scalability of SCSHEP+ and SCSHEV+ by randomly sampling between 20% and 100% of the edges from the original graph. Fig. 12 reports the results on the Amazon and Aminer datasets with  $s=20$ . As the graph size increases, the running time also increases accordingly.

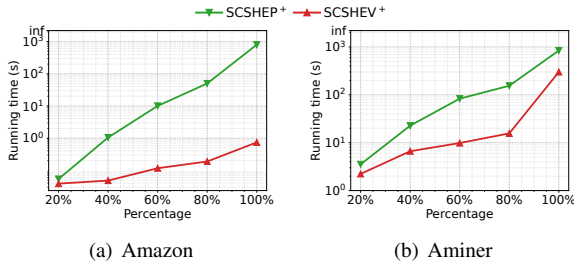


Fig. 12. Scalability

**Peak memory.** The peak memory usage of the proposed algorithm is shown in Tal. VII. The results show that the peak memory does not grow monotonically with the size bound.

Instead, the memory stabilizes once effective reduction rules is triggered during the branch-and-bound search.

TABLE VII  
AVERAGE PEAK MEMORY(MB) OF SCSHEV+

| Dataset | s=10   | s=20   | s=30   | s=40   | s=50   |
|---------|--------|--------|--------|--------|--------|
| Amazon  | 95.88  | 95.88  | 95.88  | 95.88  | 95.92  |
| Aminer  | 847.01 | 960.26 | 982.07 | 982.14 | 922.58 |

**Different meta-paths.** For Amazon, we selected the meta-paths “item, view, item”, “view, item, view”, “user, item, user” and “user, item, view, item, user”. For Aminer, the chosen meta-paths are “author, paper, author”, “paper, author, paper”, “author, paper, venue, paper, author” and “paper, venue, paper”, with the results shown in Fig. 13. It is evident that some meta-paths do not need exhaustive search and can be efficiently handled using heuristics, such as “paper, venue, paper” on Aminer.

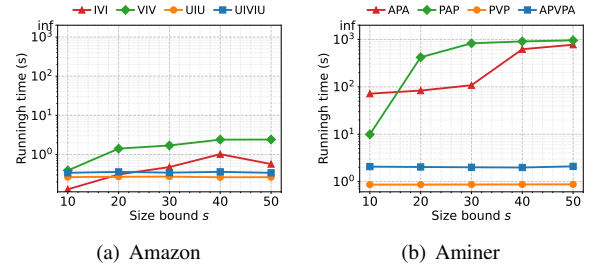


Fig. 13. Different meta-paths

**Different length of meta-path.** The results for meta-paths of different lengths are shown in Fig. 14. For shorter meta-paths, exhaustive search dominates the overall running time. For longer meta-paths, constructing  $G_P$  may lead to very dense graphs, and in such cases, the heuristic effectively handles the computation, preventing triangle enumeration or  $G_P$  construction from dominating the running time.

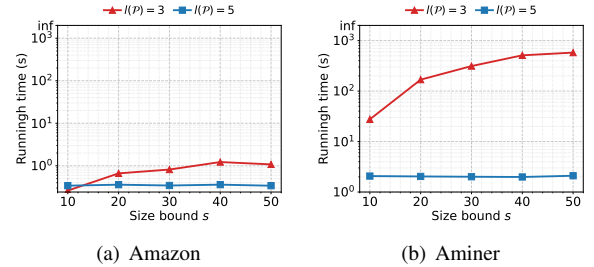


Fig. 14. Different length of meta-paths

**Different Dominance.** Fig. 15 shows the results for different dominance strategies. Triangle dominance is more efficient than edge dominance, even though it is more complex. This indicates that exploring branches dominated by triangles rather than edges is more costly than the difference in cost between finding the triangle-dominant scope and the edge-dominant scope.

**Different numbers of heuristic iterations** Fig. 16 shows the efficiency under different numbers of heuristic iterations. The results show that the efficiency remains stable and does not fluctuate significantly.

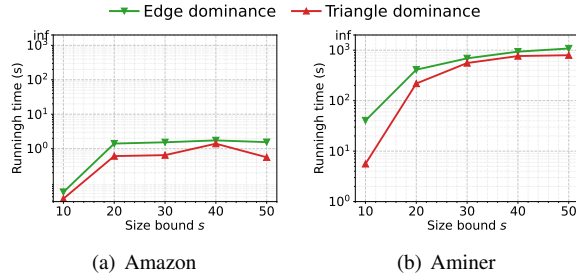


Fig. 15. Different dominance

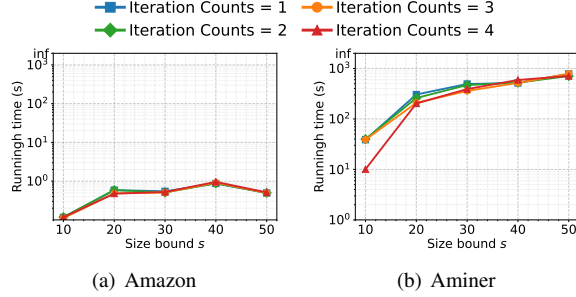


Fig. 16. Different iteration counts of heuristic

## VI. RELATED WORKS

### Size-bounded community search in homogeneous graphs.

In homogeneous graphs, many problems are related to community size, which can be broadly classified into the following categories. First, some studies impose explicit size constraints and aim to find the most cohesive communities satisfying conditions. For example, in the  $k$ -core model, the goal is to maximise  $k$  under a given size bound. [23] is the first to introduce an upper bound on community size, proposing two greedy algorithms to find the near-optimal minimum-degree subgraph. Recently, [31] extends this by introducing a size range  $[l, h]$  and proposes both an improved greedy strategy and an exact algorithm. [34] follows a similar approach but adopts the  $k$ -truss model instead of the  $k$ -core. Second, some approaches fix cohesiveness and aim to find the smallest possible community satisfying it. [15] develops the PSA algorithm based on the  $k$ -core, while [39] focuses on minimising  $k$ -truss-based communities. Finally, some studies [17] jointly enforce both size and cohesiveness constraints (without necessarily maximising  $k$  or minimising size).

**Community search in HINs.** [8] categorises heterogeneous graphs into two types: bipartite graphs and general HINs. In recent years, bipartite graphs have gained increasing attention, leading to the development of various cohesive subgraph models, including the  $(\alpha, \beta)$ -core [16],  $k$ -bitruss [28], and biclique [4], [20], [33]. Based on these models, numerous community search methods have been proposed [1], [26], [27]. For general HINs, several models have been introduced to support community search, such as the  $(k, \mathcal{P})$ -core [9],  $(k, \mathcal{P})$ -truss [31], motif-clique [10], [38], relational community [11], [18], and densest subgraph [3]. In addition, some methods focus on extra-specific objectives like significance [19] and influence [37].

**Size-bounded community search in HINs.** So far, no work has addressed size-bounded community search in general

HINs. A few studies [35], [36] have considered size-bounded community search in bipartite graphs. Notably, [35] is primarily based on the  $(\alpha, \beta)$ -core model. Since the model involves two cohesiveness parameters, it is difficult to compare the cohesiveness of different cores (e.g., a  $(4, 2)$ -core vs. a  $(3, 3)$ -core). As a result, it treats cohesiveness as a constraint rather than an optimisation objective.

## VII. CONCLUSION

In this paper, we introduce the most cohesive size-bounded community search problem for HIN data, which is known to be NP-hard. To tackle this problem, we develop the kcB&B framework, which naturally generates  $s$  node sets with a complexity of  $\mathcal{O}^*(\binom{n}{s})$ . Based on this framework, two exact algorithms are proposed that enumerate edge sets and node sets, respectively. To enhance efficiency, we first develop a polynomial-time algorithm to find a feasible solution as a global lower bound by leveraging the properties of HINs. Additionally, based on triangles, we develop bounding and reduction techniques for both edge-based and node-based approaches, which, when combined with the lower bound, can significantly reduce the search space. In addition, kcB&B enables early termination and branching strategies, further enhancing overall efficiency. Extensive experiments validate the effectiveness and efficiency of the proposed methods.

## AI-GENERATED CONTENT ACKNOWLEDGEMENT

No generative AI tools were used in the preparation of this manuscript.

## REFERENCES

- [1] Aman Abidi, Lu Chen, Rui Zhou, and Chengfei Liu. Searching personalized  $k$ -wing in bipartite graphs. *IEEE Transactions on Knowledge and Data Engineering*, 35(8):8515–8528, 2022.
- [2] Ahmed Al-Baghdadi and Xiang Lian. Topic-based community search over spatial-social networks. *Proceedings of the VLDB Endowment*, 13(12):2104–2117, 2020.
- [3] Lu Chen, Chengfei Liu, Rui Zhou, Kewen Liao, Jiajie Xu, and Jianxin Li. Densest multipartite subgraph search in heterogeneous information networks. *Proceedings of the VLDB Endowment*, 17(4):699–711, 2023.
- [4] Lu Chen, Chengfei Liu, Rui Zhou, Jiajie Xu, and Jianxin Li. Efficient exact algorithms for maximum balanced biclique search in bipartite graphs. In *Proceedings of the 2021 International Conference on Management of Data*, pages 248–260. Association for Computing Machinery, 2021.
- [5] Jonathan Cohen. Trusses: Cohesive subgraphs for social network analysis. *National security agency technical report*, 16(3.1):1–29, 2008.
- [6] Marcus T Ditttrich, Gunnar W Klau, Andreas Rosenwald, Thomas Dandekar, and Tobias Müller. Identifying functional modules in protein-protein interaction networks: an integrated exact approach. *Bioinformatics*, 24(13):i223–i231, 2008.
- [7] Yixiang Fang, CK Cheng, S Luo, J Hu, and X Li. Effective community search over large spatial graphs. *Proceedings of the VLDB Endowment*, 10(6):709–720, 2017.
- [8] Yixiang Fang, Kai Wang, Xuemin Lin, and Wenjie Zhang. Cohesive subgraph search over big heterogeneous information networks: Applications, challenges, and solutions. In *Proceedings of the 2021 International Conference on Management of Data*, pages 2829–2838. Association for Computing Machinery, 2021.
- [9] Yixiang Fang, Yixiang Yang, Wenjie Zhang, Xuemin Lin, and Xin Cao. Effective and efficient community search over large heterogeneous information networks. *Proceedings of the VLDB Endowment*, 13(6):854–867, 2020.
- [10] Jiafeng Hu, Reynold Cheng, Kevin Chen-Chuan Chang, Aravind Sankar, Yixiang Fang, and Brian YH Lam. Discovering maximal motif cliques in large heterogeneous information networks. In *2019 IEEE 35th International Conference on Data Engineering*, pages 746–757. IEEE, 2019.
- [11] Xun Jian, Yue Wang, and Lei Chen. Effective and efficient relational community detection and search in large dynamic heterogeneous information networks. *Proceedings of the VLDB Endowment*, 13(10):1723–1736, 2020.
- [12] Yangqin Jiang, Yixiang Fang, Chenhao Ma, Xin Cao, and Chunshan Li. Effective community search over large star-schema heterogeneous information networks. *Proceedings of the VLDB Endowment*, 15(11):2307–2320, 2022.
- [13] Donald E. Knuth. *The Art of Computer Programming: Combinatorial Algorithms, Part 1*. Addison-Wesley Professional, 1st edition, 2011.
- [14] Theodoros Lappas, Kun Liu, and Evimaria Terzi. Finding a team of experts in social networks. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 467–476. Association for Computing Machinery, 2009.
- [15] Conggai Li, Fan Zhang, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. Efficient progressive minimum  $k$ -core search. *Proceedings of the VLDB Endowment*, 13(3):361–374, 2020.
- [16] Boge Liu, Long Yuan, Xuemin Lin, Lu Qin, Wenjie Zhang, and Jingren Zhou. Efficient  $(\alpha, \beta)$ -core computation in bipartite graphs. *The VLDB Journal*, 29(5):1075–1099, 2020.
- [17] Boge Liu, Fan Zhang, Wenjie Zhang, Xuemin Lin, and Ying Zhang. Efficient community search with size constraint. In *2021 IEEE 37th International Conference on Data Engineering*, pages 97–108. IEEE, 2021.
- [18] Xuan Liu, Lu Chen, Chengfei Liu, and Rui Zhou. Searching society over large heterogeneous information networks. In *2025 IEEE 41st International Conference on Data Engineering*, pages 1402–1414. IEEE, 2025.
- [19] Yanghao Liu, Fangda Guo, Bingbing Xu, Peng Bao, Huawei Shen, and Xueqi Cheng. Sach: significant-attributed community search in heterogeneous information networks. In *2024 IEEE 40th International Conference on Data Engineering*, pages 3283–3296, 2024.
- [20] Bingqing Lyu, Lu Qin, Xuemin Lin, Ying Zhang, Zhengping Qian, and Jingren Zhou. Maximum biclique search at billion scale. *Proceedings of the VLDB Endowment*, 13(9):1359–1372, 2020.
- [21] Fragkiskos D Malliaros, Christos Giatsidis, Apostolos N Papadopoulos, and Michalis Vazirgiannis. The core decomposition of networks: Theory, algorithms and applications. *The VLDB Journal*, 29(1):61–92, 2020.
- [22] Stephen B Seidman. Network structure and minimum degree. *Social networks*, 5(3):269–287, 1983.
- [23] Mauro Sozio and Aristides Gionis. The community-search problem and how to plan a successful cocktail party. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 939–948. Association for Computing Machinery, 2010.
- [24] Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S Yu, and Tianyi Wu. Paths: Meta path-based top- $k$  similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment*, 4(11):992–1003, 2011.
- [25] Jia Wang and James Cheng. Truss decomposition in massive networks. *Proceedings of the VLDB Endowment*, 5(9):812–823, 2012.
- [26] Kai Wang, Wenjie Zhang, Xuemin Lin, Ying Zhang, Lu Qin, and Yuting Zhang. Efficient and effective community search on large-scale bipartite graphs. In *2021 IEEE 37th International Conference on Data Engineering*, pages 85–96. IEEE, 2021.
- [27] Kai Wang, Wenjie Zhang, Ying Zhang, Lu Qin, and Yuting Zhang. Discovering significant communities on bipartite graphs: An index-based approach. *IEEE Transactions on Knowledge and Data Engineering*, 35(3):2471–2485, 2021.
- [28] Yue Wang, Ruiqi Xu, Xun Jian, Alexander Zhou, and Lei Chen. Towards distributed bitruss decomposition on bipartite graphs. *Proceedings of the VLDB Endowment*, 15(9):1889–1901, 2022.
- [29] Tianyang Xu, Zhao Lu, and Yuanyuan Zhu. Efficient triangle-connected truss community search in dynamic graphs. *Proceedings of the VLDB Endowment*, 16(3):519–531, 2022.
- [30] Yixiang Yang, Yixiang Fang, Xuemin Lin, and Wenjie Zhang. Effective and efficient truss computation over large heterogeneous information networks. In *2020 IEEE 36th International Conference on Data Engineering*, pages 901–912. IEEE, 2020.
- [31] Kai Yao and Lijun Chang. Efficient size-bounded community search over large networks. *Proceedings of the VLDB Endowment*, 14(8):1441–1453, 2021.
- [32] Xiao Yu, Xiang Ren, Yizhou Sun, Quanquan Gu, Bradley Sturt, Urvashi Khandelwal, Brandon Norick, and Jiawei Han. Personalized entity recommendation: A heterogeneous information network approach. In *Proceedings of the 7th ACM international conference on Web search and data mining*, pages 283–292. Association for Computing Machinery, 2014.
- [33] Xintong Yu, Rui Zhou, Xiaofan Li, Lu Chen, and Chengfei Liu. Finding a summary for all maximal bicliques. In *2025 IEEE 41st International Conference on Data Engineering*, pages 3984–3997. IEEE, 2025.
- [34] Fan Zhang, Haicheng Guo, Dian Ouyang, Shiyu Yang, Xuemin Lin, and Zhihong Tian. Size-constrained community search on large networks: An effective and efficient solution. *IEEE Transactions on Knowledge and Data Engineering*, 36(1):356–371, 2023.
- [35] Yuting Zhang, Kai Wang, Wenjie Zhang, Wei Ni, and Xuemin Lin. Size-bounded community search over large bipartite graph. In *Proceedings of the 27th International Conference on Extending Database Technology*, pages 320–331, 2024.
- [36] Keqi Zhou, Junchang Xin, Jinyi Chen, Xian Zhang, Beibei Wang, and Zhiqiong Wang. Effective and efficient community search with size constraint on bipartite graphs. *Information Sciences*, 647:119511, 2023.
- [37] Yingli Zhou, Yixiang Fang, Wensheng Luo, and Yunming Ye. Influential community search over large heterogeneous information networks. *Proceedings of the VLDB Endowment*, 16(8):2047–2060, 2023.
- [38] Yingli Zhou, Yixiang Fang, Chenhao Ma, Tianci Hou, and Xin Huang. Efficient maximal motif-clique enumeration over large heterogeneous information networks. *Proceedings of the VLDB Endowment*, 17(11):2946–2959, 2024.
- [39] Weijie Zhu, Mengqi Zhang, Chen Chen, Xiaoyang Wang, Fan Zhang, and Xuemin Lin. Pivotal relationship identification: the  $k$ -truss minimization problem. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, pages 4874–4880. International Joint Conferences on Artificial Intelligence Organization, 2019.